

```
!pip install -q --upgrade scipy
!pip install -q monai nibabel timm tqdm scikit-learn
import numpy, scipy, monai, timm
print("All OK – restart runtime now")
```

```

_____ 62.3/62.3 kB 5.8 MB/s eta 0:00:00
_____ 35.3/35.3 MB 75.8 MB/s eta 0:00:00
_____ 1.6/1.6 MB 73.8 MB/s eta 0:00:00
All OK – restart runtime now
```

```
from google.colab import drive
drive.mount('/content/drive', force_remount=True)
```

Mounted at /content/drive

```
import shutil, zipfile, time, os

UTSW_ZIP_DRIVE = '/content/drive/MyDrive/utsw_essential.zip'
UTSW_LOCAL_ZIP = '/content/utsw_essential.zip'
UTSW_LOCAL_DIR = '/content/UTSW-Glioma'

if not os.path.exists(UTSW_LOCAL_DIR):
    print("Copying UTSW zip from Drive...")
    start = time.time()
    shutil.copy2(UTSW_ZIP_DRIVE, UTSW_LOCAL_ZIP)
    print(f"Copy done in {time.time()-start:.1f}s")

    print("Extracting...")
    start = time.time()
    with zipfile.ZipFile(UTSW_LOCAL_ZIP, 'r') as zf:
        zf.extractall(UTSW_LOCAL_DIR)
    print(f"Extract done in {time.time()-start:.1f}s")

    os.remove(UTSW_LOCAL_ZIP)

    folders = [f for f in os.listdir(UTSW_LOCAL_DIR) if f.startswith('BT')]
    print(f"Extracted {len(folders)} patient folders")
else:
    folders = [f for f in os.listdir(UTSW_LOCAL_DIR) if f.startswith('BT')]
```

```
print(f"Already local – {len(folders)} patient folders")
```

```
UTSW_DIR = UTSW_LOCAL_DIR
print(f"UTSW_DIR = {UTSW_DIR}")
```

```
Copying UTSW zip from Drive...
Copy done in 77.4s
Extracting...
Extract done in 20.7s
Extracted 362 patient folders
UTSW_DIR = /content/UTSW-Glioma
```

```
from google.colab import drive
drive.mount('/content/drive')

import os, torch

# — Check checkpoint —————
FINETUNE_CKPT = '/content/drive/MyDrive/checkpoints/swinunetr_ucsf_finetuned.pt'

if os.path.exists(FINETUNE_CKPT):
    ckpt = torch.load(FINETUNE_CKPT, map_location='cpu', weights_only=False)
    print(f"✓ Checkpoint found")
    print(f"  Epoch:      {ckpt['epoch']}")
    print(f"  Val Dice:   {ckpt['val_dice']:.4f}")
else:
    print(f"✗ Checkpoint NOT found at {FINETUNE_CKPT}")

# — Check UCSF dataset —————
UCSF_DIR = '/content/drive/MyDrive/UCSF-PDGM-v5'
UCSF_CSV = '/content/drive/MyDrive/UCSF-PDGM-metadata_v5.csv'

if os.path.exists(UCSF_DIR):
    folders = [f for f in os.listdir(UCSF_DIR) if os.path.isdir(os.path.join(UCSF_DIR, f))]
    print(f"\n✓ UCSF dataset found")
    print(f"  Patient folders: {len(folders)}")
else:
    print(f"\n✗ UCSF dataset NOT found at {UCSF_DIR}")

if os.path.exists(UCSF_CSV):
    import pandas as pd
    df = pd.read_csv(UCSF_CSV)
    print(f"✓ Metadata CSV found – {len(df)} rows")
```

```

else:
    print(f"x Metadata CSV NOT found at {UCSF_CSV}")

# — Check feature cache —————
CACHE_DIR = '/content/drive/MyDrive/checkpoints/feature_cache_3d_ucsf_finetuned'

if os.path.exists(CACHE_DIR):
    files = [f for f in os.listdir(CACHE_DIR) if f.endswith('.pt')]
    print(f"\n✓ Feature cache found")
    print(f"  Files: {len(files)}")
else:
    print(f"\nx Feature cache NOT found at {CACHE_DIR}")

# — Write test —————
test_path = '/content/drive/MyDrive/checkpoints/write_test.txt'
try:
    with open(test_path, 'w') as f:
        f.write('test')
    os.remove(test_path)
    print(f"\n✓ Drive is writable")
except Exception as e:
    print(f"\nx Drive is READ-ONLY: {e}")

```

Mounted at /content/drive

✓ Checkpoint found

Epoch: 97

Val Dice: 0.8846

✓ UCSF dataset found

Patient folders: 501

✓ Metadata CSV found – 501 rows

✓ Feature cache found

Files: 487

✓ Drive is writable

```

# =====
# CELL 2 – Mount Drive + Config
# =====

```

```

from google.colab import drive

```

```

drive.mount('/content/drive')

import os, torch

class Config:
    UCSF_DIR   = '/content/drive/MyDrive/UCSF-PDGM-v5'
    UCSF_CSV   = '/content/drive/MyDrive/UCSF-PDGM-metadata_v5.csv'
    CKPT_DIR   = '/content/drive/MyDrive/checkpoints'
    WEIGHTS    = '/content/drive/MyDrive/checkpoints/swinunetr_brats21_fold1.pt'

    SPATIAL_SIZE = (128, 128, 128)
    VOXEL_SPACING = (1.0, 1.0, 1.0)

    IN_CHANNELS  = 4
    OUT_CHANNELS = 3
    FEATURE_SIZE = 48
    ENCODER_DIM   = 768      # Stage 4 bottleneck channels

    TTA_RUNS = 5

    MORPH_FEATURES = 8
    FUSED_DIM      = 512
    SEG_THRESHOLD  = 0.3

    # — Ablation control —————
    # 'A' encoder only | 'B' fixed gate | 'C' G=conf | 'D' proposed
    GATE_MODE = 'D'

    # — Backbone control —————
    FREEZE_BACKBONE = True    # set False for fine-tune ablation

    BATCH_SIZE = 8            # bigger batch – features are tiny now
    LR          = 3e-4
    CLF_EPOCHS  = 60
    DEVICE      = 'cuda' if torch.cuda.is_available() else 'cpu'

cfg = Config()
os.makedirs(cfg.CKPT_DIR, exist_ok=True)
print(f"Device: {cfg.DEVICE}")
print(f"Gate mode: {cfg.GATE_MODE}  Backbone frozen: {cfg.FREEZE_BACKBONE}")
if cfg.DEVICE == 'cpu':
    print("WARNING: GPU not enabled")

```

```

def csv_id_to_file_id(csv_id):
    prefix, num = csv_id.rsplit('-', 1)
    return f'{prefix}-{num.zfill(4)}'

def get_patient_paths(csv_id, ucsf_dir):
    file_id = csv_id_to_file_id(csv_id)
    folder_path = os.path.join(ucsf_dir, f'{file_id}_nifti')
    return {
        'folder': folder_path,
        't1': os.path.join(folder_path, f'{file_id}_T1_bias.nii.gz'),
        't1c': os.path.join(folder_path, f'{file_id}_T1c_bias.nii.gz'),
        't2': os.path.join(folder_path, f'{file_id}_T2_bias.nii.gz'),
        'flair': os.path.join(folder_path, f'{file_id}_FLAIR_bias.nii.gz'),
        'label': os.path.join(folder_path, f'{file_id}_tumor_segmentation.nii.gz'),
    }

def verify_patient_files(csv_id, ucsf_dir):
    paths = get_patient_paths(csv_id, ucsf_dir)
    missing = [k for k, v in paths.items()
                if k != 'folder' and not os.path.exists(v)]
    return (len(missing) == 0), missing

print("Config and helpers loaded")

```

```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True)
Device: cuda
Gate mode: D Backbone frozen: True
Config and helpers loaded

```

```

# =====
# CELL 2B – Restore slim cache from Drive backup (fast session start)
# Run this every new session instead of rebuilding from full cache
# =====
import shutil, time, os

SLIM_CACHE_DRIVE = '/content/drive/MyDrive/checkpoints/feature_cache_slim_ucsf'
SLIM_CACHE_LOCAL = '/content/feature_cache_slim_ucsf'

if not os.path.exists(SLIM_CACHE_LOCAL):
    if os.path.exists(SLIM_CACHE_DRIVE):

```

```

    print("Copying slim cache from Drive...")
    start = time.time()
    shutil.copytree(SLIM_CACHE_DRIVE, SLIM_CACHE_LOCAL)
    print(f"Done in {time.time()-start:.1f}s - "
          f"{len(os.listdir(SLIM_CACHE_LOCAL))} files")
else:
    print("No Drive backup found – run Cell 11 + Cell 12 to build fresh")
else:
    print(f"Already local – {len(os.listdir(SLIM_CACHE_LOCAL))} files")

SLIM_CACHE = SLIM_CACHE_LOCAL
print(f"SLIM_CACHE = {SLIM_CACHE}")

```

```

Copying slim cache from Drive...
Done in 11.6s – 487 files
SLIM_CACHE = /content/feature_cache_slim_ucsf

```

```

# add zip of ucsf to local storage from drive
import shutil, zipfile, time, os

ZIP_PATH   = '/content/drive/MyDrive/checkpoints/ucsf_essential.zip'
LOCAL_ZIP  = '/content/ucsf_essential.zip'
LOCAL_DIR  = '/content/UCSF-PDGM-v5'

print("Copying single zip file from Drive...")
start = time.time()
shutil.copy2(ZIP_PATH, LOCAL_ZIP)
print(f"Copy done in {time.time()-start:.1f}s")

print("Extracting...")
start = time.time()
with zipfile.ZipFile(LOCAL_ZIP, 'r') as zf:
    zf.extractall(LOCAL_DIR)
print(f"Extract done in {time.time()-start:.1f}s")

os.remove(LOCAL_ZIP)
cfg.UCSF_DIR = LOCAL_DIR
print(f"cfg.UCSF_DIR = {cfg.UCSF_DIR}")

```

```

Copying single zip file from Drive...
Copy done in 139.8s
Extracting...

```

```
Extract done in 19.7s
cfg.UCSF_DIR = /content/UCSF-PDGM-v5
```

```
print(cfg.UCSF_DIR)
print(os.path.exists(cfg.UCSF_DIR))
```

```
/content/UCSF-PDGM-v5
True
```

```
print(cfg.DEVICE)
print(next(swinunetr.parameters()).device if 'swinunetr' in dir() else "not built yet")
```

```
cuda
not built yet
```

```
# =====
# CELL 3 – Imports
# =====

import numpy as np
import pandas as pd
import torch.nn as nn
import torch.nn.functional as F
import nibabel as nib
from scipy import ndimage
from scipy.ndimage import distance_transform_edt, binary_erosion
from tqdm import tqdm
import monai

from monai.transforms import (
    Compose, LoadImaged, EnsureChannelFirstd, ConcatItemsd,
    Orientationd, Spacingd, NormalizeIntensityd, CropForegroundd,
    ResizeWithPadOrCropd, RandFlipd, RandRotate90d,
    RandShiftIntensityd, ToTensord, MapLabelValued
)
from monai.networks.nets import SwinUNETR
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, roc_auc_score, roc_curve, confusion_matrix,
    ConfusionMatrixDisplay
)
```

```
print(f'MONAI: {monai.__version__} Torch: {torch.__version__}')
print("All imports OK")
```

```
MONAI: 1.6.0 Torch: 2.11.0+cu128
All imports OK
```

```
# =====
# CELL 4 – UCSF data loading
# =====

def build_ucsf_dicts(csv_path, ucsf_dir):
    df = pd.read_csv(csv_path)
    data_dicts, missing_count = [], 0

    for _, row in df.iterrows():
        csv_id = row['ID']
        paths = get_patient_paths(csv_id, ucsf_dir)
        if not all(os.path.exists(paths[k])
                   for k in ['t1', 't1c', 't2', 'flair', 'label']):
            missing_count += 1
            continue

        idh_raw = str(row['IDH']).strip().lower()
        grade_raw = str(row['WHO CNS Grade']).strip()
        mgmt_raw = str(row['MGMT status']).strip().lower()

        idh = (0.0 if idh_raw == 'wildtype'
              else float('nan') if idh_raw in ['', 'nan', 'unknown']
              else 1.0)
        # Binary grade: low (2/3) = 0, high (4) = 1
        grade = (1.0 if grade_raw == '4'
                else 0.0 if grade_raw in ['2', '3']
                else float('nan'))
        mgmt = (1.0 if mgmt_raw == 'positive'
              else 0.0 if mgmt_raw == 'negative'
              else float('nan'))

        data_dicts.append({
            'patient_id': csv_id,
```

```

        't1': paths['t1'], 't1c': paths['t1c'],
        't2': paths['t2'], 'flair': paths['flair'],
        'label': paths['label'],
        'idh': idh, 'grade': grade, 'mgmt': mgmt,
    })

```

```

print(f"Patients loaded: {len(data_dicts)}")
print(f"Patients skipped: {missing_count}")
idh_0 = sum(1 for d in data_dicts if d['idh']==0.0)
idh_1 = sum(1 for d in data_dicts if d['idh']==1.0)
g_lo = sum(1 for d in data_dicts if d['grade']==0.0)
g_hi = sum(1 for d in data_dicts if d['grade']==1.0)
m_neg = sum(1 for d in data_dicts if d['mgmt']==0.0)
m_pos = sum(1 for d in data_dicts if d['mgmt']==1.0)
print(f"\nIDH: {idh_0} wt / {idh_1} mut")
print(f"Grade: {g_lo} low / {g_hi} high")
print(f"MGMT: {m_neg} neg / {m_pos} pos")
return data_dicts

```

```
def get_ucsf_transforms(mode='train'):
```

```

    base = [
        LoadImaged(keys=['t1', 't1c', 't2', 'flair', 'label']),
        EnsureChannelFirstd(keys=['t1', 't1c', 't2', 'flair', 'label']),
        ConcatItemsd(keys=['t1', 't1c', 't2', 'flair'], name='image'),
        Orientationd(keys=['image', 'label'], axcodes='RAS'),
        Spacingd(keys=['image', 'label'], pixdim=config.VOXEL_SPACING,
                  mode=('bilinear', 'nearest')),
        NormalizeIntensityd(keys='image', nonzero=True, channel_wise=True),
        CropForegroundd(keys=['image', 'label'], source_key='image'),
        ResizeWithPadOrCropd(keys=['image', 'label'],
                              spatial_size=config.SPATIAL_SIZE),
        MapLabelValued(keys=['label'], orig_labels=[0,1,2,4],
                       target_labels=[0,1,2,3]),
        ToTensord(keys=['image', 'label'])
    ]
    aug = [
        RandFlipd(keys=['image', 'label'], prob=0.5, spatial_axis=0),
        RandFlipd(keys=['image', 'label'], prob=0.5, spatial_axis=1),
        RandFlipd(keys=['image', 'label'], prob=0.5, spatial_axis=2),
        RandRotate90d(keys=['image', 'label'], prob=0.5, max_k=3),
        RandShiftIntensityd(keys=['image'], offsets=0.1, prob=0.5),

```

```

]
return Compose(base[:-1]+aug+[base[-1]] if mode=='train' else base)

def stratified_split(all_dicts, val_split=0.15, test_split=0.15, seed=42):
    """
    Stratifies on a combined (IDH, Grade) key so both labels
    are proportionally represented in train/val/test.
    MGMT is not included in the key (would create too many
    small groups) but tends to follow along reasonably.
    """
    import random
    rng = random.Random(seed)

    def key(d):
        idh = 'x' if np.isnan(d['idh']) else str(int(d['idh']))
        grd = 'x' if np.isnan(d['grade']) else str(int(d['grade']))
        return (idh, grd)

    groups = {}
    for d in all_dicts:
        groups.setdefault(key(d), []).append(d)

    train_d, val_d, test_d = [], [], []
    for k, group in groups.items():
        shuffled = group[:]
        rng.shuffle(shuffled)
        n = len(shuffled)
        n_test = int(n * test_split)
        n_val = int(n * val_split)
        test_d += shuffled[:n_test]
        val_d += shuffled[n_test:n_test+n_val]
        train_d += shuffled[n_test+n_val:]

    return train_d, val_d, test_d
print("Data loading functions defined")

```

Data loading functions defined

```

# =====
# CELL 4B – UTSW data loading

```

```

# =====
UTSW_DIR = '/content/drive/MyDrive/UTSW-Glioma'
UTSW_TSV = '/content/drive/MyDrive/UTSW_Glioma_Metadata-2-1.tsv'
UTSW_CACHE_DIR = '/content/drive/MyDrive/checkpoints/feature_cache_3d_utsw'

def build_utsw_dicts(tsv_path, utsw_dir, refined_only=True):
    df = pd.read_csv(tsv_path, sep='\t')
    if refined_only:
        df = df[df['Manually Refined Segmentation'] == 'Yes']
        print(f"Using {len(df)} manually refined patients")

    data_dicts, missing_count = [], 0
    for _, row in df.iterrows():
        pid = row['Subject ID']
        folder = os.path.join(utsw_dir, pid)
        paths = {
            't1': os.path.join(folder, 'brain_t1_ants.nii.gz'),
            't1c': os.path.join(folder, 'brain_t1ce_ants.nii.gz'),
            't2': os.path.join(folder, 'brain_t2_ants.nii.gz'),
            'flair': os.path.join(folder, 'brain_fl_ants.nii.gz'),
            'label': os.path.join(folder, 'tumorseg_resampled.nii.gz'),
        }
        if not all(os.path.exists(v) for v in paths.values()):
            missing_count += 1
            continue

        idh_raw = str(row['IDH']).strip().lower()
        grade_raw = str(row['Tumor Grade']).strip().replace('.0', '')
        mgmt_raw = str(row['MGMT']).strip().lower()

        idh = (0.0 if idh_raw == 'wild type'
              else 1.0 if idh_raw == 'mutated'
              else float('nan'))
        grade = (1.0 if grade_raw == '4'
                else 0.0 if grade_raw in ['2', '3']
                else float('nan'))
        mgmt = (1.0 if mgmt_raw == 'methylated'
                else 0.0 if mgmt_raw == 'unmethylated'
                else float('nan'))

        data_dicts.append({
            'patient_id': pid,

```

```

    **paths,
    'idh': idh, 'grade': grade, 'mgmt': mgmt,
})

```

```

print(f"Patients loaded: {len(data_dicts)}")
print(f"Patients skipped: {missing_count}")
idh_0 = sum(1 for d in data_dicts if d['idh']==0.0)
idh_1 = sum(1 for d in data_dicts if d['idh']==1.0)
g_lo = sum(1 for d in data_dicts if d['grade']==0.0)
g_hi = sum(1 for d in data_dicts if d['grade']==1.0)
m_neg = sum(1 for d in data_dicts if d['mgmt']==0.0)
m_pos = sum(1 for d in data_dicts if d['mgmt']==1.0)
print(f"IDH: {idh_0} wt / {idh_1} mut")
print(f"Grade: {g_lo} low / {g_hi} high")
print(f"MGMT: {m_neg} neg / {m_pos} pos (note: {len(data_dicts)-m_neg-m_pos} missing)")
return data_dicts

```

```
print("UTSW data loading functions defined")
```

```
UTSW data loading functions defined
```

```

# =====
# CELL 4C – UTSW label resampling (run ONCE after download)
# =====
import nibabel as nib
import nibabel.processing as nbproc

def preprocess_utsw_labels(utsw_dir):
    patients = sorted([p for p in os.listdir(utsw_dir)
                        if os.path.isdir(os.path.join(utsw_dir, p))
                        and p.startswith('BT')])
    print(f"Found {len(patients)} patient folders")
    done, skipped, failed = 0, 0, 0

    for pid in tqdm(patients, desc="Resampling labels"):
        folder = os.path.join(utsw_dir, pid)
        t1_path = os.path.join(folder, 'brain_t1_ants.nii.gz')
        seg_path = os.path.join(folder, 'tumorseg_manual_correction.nii.gz')
        out_path = os.path.join(folder, 'tumorseg_resampled.nii.gz')

        if os.path.exists(out_path):
            skipped += 1

```

```

        continue
    if not os.path.exists(t1_path) or not os.path.exists(seg_path):
        print(f"Missing files for {pid}")
        failed += 1
        continue
    try:
        t1 = nib.load(t1_path)
        seg = nib.load(seg_path)
        resampled = nbproc.resample_from_to(seg, t1, order=0)
        labels = set(resampled.get_fdata().astype(int).flat)
        labels.discard(0)
        if not labels.issubset({1, 2, 4}):
            print(f"Unexpected labels for {pid}: {labels}")
            failed += 1
            continue
        nib.save(resampled, out_path)
        done += 1
    except Exception as e:
        print(f"Failed {pid}: {e}")
        failed += 1

    print(f"\nDone: {done} Skipped (exist): {skipped} Failed: {failed}")

# Run once – takes ~5 minutes, creates tumorseg_resampled.nii.gz for each patient
preprocess_uts_w_labels(UTSW_DIR)

```

```

cfg.UCSF_DIR = '/content/UCSF-PDGM-v5'

```

```

for item in all_dicts:
    paths = get_patient_paths(item['patient_id'], cfg.UCSF_DIR)
    img = nib.load(paths['t1'])
    lab = nib.load(paths['label'])
    if img.shape != lab.shape or not np.allclose(img.affine, lab.affine, atol=1e-3):
        print(item['patient_id'], img.shape, lab.shape,
              np.allclose(img.affine, lab.affine, atol=1e-3))

```

```

# =====
# CELL 5 – SwinUNETR with bottleneck feature extraction
# =====

```

```

def build_swinunetr(weights_path=None):
    model = SwinUNETR(
        in_channels=cfg.IN_CHANNELS,
        out_channels=cfg.OUT_CHANNELS,
        feature_size=cfg.FEATURE_SIZE,
        use_checkpoint=True,
    ).to(cfg.DEVICE)

    if weights_path and os.path.exists(weights_path):
        print(f"Loading: {weights_path}")
        ckpt = torch.load(weights_path, map_location=cfg.DEVICE,
                           weights_only=False)
        state = ckpt['state_dict']

        # Strip common prefixes that cause mismatch
        cleaned = {}
        for k, v in state.items():
            new_k = k
            for prefix in ['module.', 'model.', 'backbone.']:
                if new_k.startswith(prefix):
                    new_k = new_k[len(prefix):]
            cleaned[new_k] = v

        model_dict = model.state_dict()

        # Separate matched from unmatched for diagnostics
        matched = {k:v for k,v in cleaned.items()
                    if k in model_dict and model_dict[k].shape==v.shape}
        unmatched = {k:v for k,v in cleaned.items()
                     if k not in model_dict}
        wrong_shape = {k:v for k,v in cleaned.items()
                        if k in model_dict and model_dict[k].shape!=v.shape}

        model_dict.update(matched)
        model.load_state_dict(model_dict)

        print(f"Epoch: {ckpt['epoch']} Dice: {ckpt['best_acc']:.4f}")
        print(f"Matched:      {len(matched)}/{len(model_dict)}")
        print(f"Unmatched:    {len(unmatched)}")
        print(f"Wrong shape: {len(wrong_shape)}")

```

```

    if len(matched) < len(model_dict) * 0.9:
        print("WARNING: fewer than 90% of layers loaded")
        print("Sample unmatched keys:")
        for k in list(unmatched.keys())[:5]:
            print(f"  checkpoint: {k}")
        print("Sample model keys:")
        for k in list(model_dict.keys())[:5]:
            print(f"  model:      {k}")

if cfg.FREEZE_BACKBONE:
    for p in model.parameters():
        p.requires_grad = False
    model.eval()
    print("Backbone FROZEN")

print(f"Parameters: {sum(p.numel() for p in model.parameters())/1e6:.1f}M")
return model

def extract_encoder_features(model, image_tensor):
    """
    Extracts Stage 4 bottleneck features from SwinUNETR.

    SwinViT for 128^3 input returns 5 hidden states:
    [0] [B, 48, 64, 64, 64]
    [1] [B, 96, 32, 32, 32]
    [2] [B, 192, 16, 16, 16]
    [3] [B, 384, 8, 8, 8]
    [4] [B, 768, 4, 4, 4] ← bottleneck, what we use

    Returns: [768] global-average-pooled feature vector
    """
    with torch.no_grad():
        x = image_tensor.unsqueeze(0).to(cfg.DEVICE) # [1,4,128,128,128]
        hidden_states = model.swinViT(x)             # list of 5 tensors
        bottleneck     = hidden_states[4]             # [1,768,4,4,4]
        features       = bottleneck.mean(dim=[2,3,4]) # [1,768]
    return features.squeeze(0).cpu()                 # [768]

swinunetr = build_swinunetr(cfg.WEIGHTS)

```

```

Loading: /content/drive/MyDrive/checkpoints/swinunetr_brats21_fold1.pt
Epoch: 279  Dice: 0.9059
Matched:      159/159
Unmatched:    0
Wrong shape:  0
Backbone FROZEN
Parameters: 62.2M

```

```

# MANDATORY LOADER
ckpt = torch.load(os.path.join(cfg.CKPT_DIR, 'swinunetr_ucsf_finetuned.pt'),
                  map_location=cfg.DEVICE, weights_only=False)
swinunetr.load_state_dict(ckpt['state_dict'])
swinunetr.eval()
print(f"Loaded epoch {ckpt['epoch']} - Val Dice {ckpt['val_dice']:.4f}")

checksum = sum(p.sum().item() for p in swinunetr.parameters())
print(f"Checksum: {checksum:.2f}")
assert abs(checksum - (-34969.76)) < 1.0, "WRONG WEIGHTS LOADED - stop and investigate"

```

```

Loaded epoch 97 - Val Dice 0.8846
Checksum: -34969.76

```

```

# CELL 6
def tta_predict(model, image_tensor, n_runs=cfg.TTA_RUNS):
    """Returns mean_pred [3,H,W,D], conf_map [1,H,W,D], uncertainty."""
    model.eval()
    preds = []
    with torch.no_grad():
        for _ in range(n_runs):
            aug = image_tensor.clone()
            flipped_axes = []
            for axis in [1,2,3]:
                if torch.rand(1).item() > 0.5:
                    aug = torch.flip(aug, dims=[axis])
                    flipped_axes.append(axis)
            aug = aug + (torch.rand(1).item()-0.5)*0.2
            logits = model(aug.unsqueeze(0).to(cfg.DEVICE))
            probs = torch.sigmoid(logits).squeeze(0).cpu()
            # Unflip on the SAME axis index - input and output share
            # the same spatial axis positions (1,2,3)
            for axis in reversed(flipped_axes):

```

```

        probs = torch.flip(probs, dims=[axis])
        preds.append(probs)
    stack = torch.stack(preds, dim=0)
    mean_pred = stack.mean(dim=0)
    variance = stack.var(dim=0).mean(0, keepdim=True)
    vmin, vmax = variance.min(), variance.max()
    norm_var = (variance - vmin) / (vmax - vmin + 1e-8)
    return mean_pred, 1.0 - norm_var, norm_var
print("TTA function corrected and verified – same axis index for flip/unflip")

```

TTA function corrected and verified – same axis index for flip/unflip

```

# =====
# CELL 7 – Morphological features
# =====

def extract_morphological_features(mean_pred):
    et = (mean_pred[0]>0.3).numpy().astype(np.float32)
    tc = (mean_pred[1]>0.3).numpy().astype(np.float32)
    wt = (mean_pred[2]>0.3).numpy().astype(np.float32)
    edema = np.clip(wt - tc, 0, 1)

    tumor_vol, core_vol = float(wt.sum()), float(tc.sum())
    edema_vol, enh_vol = float(edema.sum()), float(et.sum())
    edema_core_ratio = edema_vol / (core_vol + 1e-6)

    eroded = ndimage.binary_erosion(wt.astype(bool))
    surface = wt.astype(bool) & ~eroded
    sa = float(surface.sum())
    compactness = tumor_vol / (sa * 1.5 + 1e-6) if sa > 0 else 0.0
    sphericity = ((np.pi * (1/3) * (6 * tumor_vol) ** (2/3)) / (sa + 1e-6))
                if (sa > 0 and tumor_vol > 0) else 0.0

    return {
        'tumor_volume': tumor_vol, 'core_volume': core_vol,
        'edema_volume': edema_vol, 'enhancing_volume': enh_vol,
        'edema_core_ratio': edema_core_ratio, 'surface_area': sa,
        'compactness': compactness, 'sphericity': sphericity,
    }

def morphology_to_tensor(d):
    t = torch.tensor(list(d.values()), dtype=torch.float32)

```

```

# Replace any inf or nan with 0 first
t = torch.nan_to_num(t, nan=0.0, posinf=0.0, neginf=0.0)

# log1p the 4 volume features (indices 0-3)
t[:4] = torch.log1p(t[:4])

# log1p surface area (index 5)
t[5] = torch.log1p(t[5])

# Clamp edema_core_ratio (index 4) – can explode when core_vol ~ 0
t[4] = torch.clamp(t[4], 0, 10)

# Clamp compactness (index 6) – worst offender, can be billions
t[6] = torch.clamp(t[6], 0, 10)

# Clamp sphericity (index 7) – theoretically 0-1 but can exceed due to voxel discretization
t[7] = torch.clamp(t[7], 0, 2)

return t

print("Morphology functions defined")

```

Morphology functions defined

```

# =====
# CELL 8 – Confidence-Gated Fusion (4 ablation modes)
# =====

class ConfidenceGatedFusion(nn.Module):
    """
    Gate modes:
    'A' encoder only – no fusion
    'B' fixed gate G=0.5
    'C' G = confidence
    'D' G = confidence × sigmoid(W_m · morph)    [proposed]
    """
    def __init__(self, encoder_dim, morph_dim, fused_dim=cfg.FUSED_DIM,
                 mode='D'):
        super().__init__()
        self.mode = mode

```

```

self.encoder_proj = nn.Sequential(
    nn.Linear(encoder_dim, fused_dim),
    nn.ReLU(), nn.Dropout(0.3),
)
self.morph_proj = nn.Sequential(
    nn.Linear(morph_dim, 256),
    nn.ReLU(), nn.Dropout(0.3),
    nn.Linear(256, fused_dim),
)
# Morphology-driven gate component (used in mode D)
self.morph_gate = nn.Linear(morph_dim, fused_dim)

def forward(self, encoder_features, morph_features, confidence_score):
    enc = self.encoder_proj(encoder_features)
    morph = self.morph_proj(morph_features)
    conf = confidence_score.view(-1, 1) # [B,1]

    if self.mode == 'A':
        # encoder only
        return enc

    if self.mode == 'B':
        # fixed equal gate
        G = 0.5
        return G * enc + (1.0 - G) * morph

    if self.mode == 'C':
        # gate = confidence only
        G = conf
        return G * enc + (1.0 - G) * morph

    # mode 'D' - proposed
    # G = confidence × sigmoid(W_m · morph)
    morph_gate = torch.sigmoid(self.morph_gate(morph_features)) # [B,fused]
    G = conf * morph_gate # [B,fused]
    return G * enc + (1.0 - G) * morph

print("ConfidenceGatedFusion defined")

```

ConfidenceGatedFusion defined

```

# =====
# CELL 9 – Classifier (encoder features + morph + confidence)
# =====

class BrainTumorClassifier(nn.Module):
    def __init__(self, encoder_dim=cfg.ENCODER_DIM,
                  morph_dim=cfg.MORPH_FEATURES,
                  fused_dim=cfg.FUSED_DIM, dropout=0.4,
                  gate_mode=cfg.GATE_MODE):
        super().__init__()
        self.fusion = ConfidenceGatedFusion(
            encoder_dim, morph_dim, fused_dim, mode=gate_mode)
        self.shared = nn.Sequential(
            nn.Linear(fused_dim, 256), nn.ReLU(), nn.Dropout(dropout),
        )
        self.head_idh = nn.Linear(256, 1)
        self.head_grade = nn.Linear(256, 1)
        self.head_mgmt = nn.Linear(256, 1)

    def forward(self, encoder_features, morph_features, confidence_score):
        fused = self.fusion(encoder_features, morph_features,
                             confidence_score)
        shared = self.shared(fused)
        return {
            'idh': self.head_idh(shared),
            'grade': self.head_grade(shared),
            'mgmt': self.head_mgmt(shared),
        }

classifier = BrainTumorClassifier().to(cfg.DEVICE)
print(f"Classifier params: "
      f"{sum(p.numel() for p in classifier.parameters())/1e6:.2f}M")
print(f"Gate mode: {cfg.GATE_MODE}")

```

```

Classifier params: 0.66M
Gate mode: D

```

```

print(f"Pre-cache checksum: {sum(p.sum().item() for p in swinunetr.parameters()):.2f}")
print(f"Expected (fine-tuned epoch 97): should differ from BraTS-only build")

```

```
Pre-cache checksum: -34969.76
Expected (fine-tuned epoch 97): should differ from BraTS-only build
```

```
print(f"Current loaded checksum: {sum(p.sum().item() for p in swinunetr.parameters()):.2f}")
print("Fine-tuned reference:      -34969.76")
print("BraTS-only reference:      -64563.11")
```

```
Current loaded checksum: -34969.76
Fine-tuned reference:    -34969.76
BraTS-only reference:    -64563.11
```

```
# =====
# CELL 9A – Fine-tune SwinUNETR on UCSF-PDGM
# Resuming from epoch 34 (Val Dice 0.8653) with adjusted LR
# =====

import gc
from monai.losses import DiceLoss
from monai.metrics import DiceMetric
from monai.utils.enums import MetricReduction

FINETUNE_EPOCHS    = 100
FINETUNE_LR        = 1e-4      # bumped from 5e-5 to escape plateau
ACCUMULATION_STEPS = 4
FINETUNE_CKPT      = os.path.join(cfg.CKPT_DIR, 'swinunetr_ucsf_finetuned.pt')
LATEST_CKPT        = os.path.join(cfg.CKPT_DIR, 'swinunetr_ucsf_latest.pt')

# — Step 1: Freeze early stages, unfreeze late stages —
def set_finetune_layers(model):
    for p in model.parameters():
        p.requires_grad = False
    for p in model.swinViT.layers3.parameters():
        p.requires_grad = True
    for p in model.swinViT.layers4.parameters():
        p.requires_grad = True
    for name, p in model.named_parameters():
        if any(k in name for k in [
            'encoder10', 'encoder1', 'encoder2',
            'encoder3', 'encoder4',
            'decoder5', 'decoder4', 'decoder3',
            'decoder2', 'decoder1', 'out'
```

```

    ]):
        p.requires_grad = True
    trainable = sum(p.numel() for p in model.parameters() if p.requires_grad)
    frozen     = sum(p.numel() for p in model.parameters() if not p.requires_grad)
    print(f"Trainable params: {trainable/1e6:.1f}M")
    print(f"Frozen params:   {frozen/1e6:.1f}M")

set_finetune_layers(swinunetr)
swinunetr.train()

# — Step 2: Define optimizer and scheduler FIRST —————
optimizer = torch.optim.AdamW(
    filter(lambda p: p.requires_grad, swinunetr.parameters()),
    lr=FINETUNE_LR, weight_decay=1e-5)

scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
    optimizer, T_max=100)

# — Step 3: Force restart from best checkpoint (epoch 34) ———
# Deliberately NOT resuming from LATEST_CKPT (the plateaued
# epoch 47-50 state) – starting fresh from the best known point
# with new LR and optimizer/scheduler state, since the old
# optimizer state was tuned for the now-abandoned LR=5e-5 run.

RESUME          = True
start_epoch     = 0
best_val_dice   = 0.0
no_improve      = 0
history         = {'train_loss': [], 'val_dice': []}

if RESUME and os.path.exists(FINETUNE_CKPT):
    ckpt = torch.load(FINETUNE_CKPT, map_location=cfg.DEVICE, weights_only=False)
    swinunetr.load_state_dict(ckpt['state_dict'])
    # Intentionally NOT loading optimizer/scheduler state –
    # fresh optimizer for the new LR regime
    best_val_dice = ckpt['val_dice']
    history       = ckpt.get('history', {'train_loss': [], 'val_dice': []})
    # Continue epoch numbering from where the best model was,
    # not from the plateaued latest checkpoint
    start_epoch   = ckpt['epoch'] + 1
    print(f"Loaded best checkpoint – epoch {ckpt['epoch']}, "
          f"Val Dice {best_val_dice:.4f}")

```

```

        print(f"Restarting training from epoch {start_epoch} "
              f"with fresh optimizer at LR={FINETUNE_LR}")
    else:
        print("Starting from scratch")

# — Step 4: DataLoaders —————
class SegDataset(torch.utils.data.Dataset):
    def __init__(self, data_dicts, transforms):
        self.data      = data_dicts
        self.transforms = transforms
    def __len__(self):
        return len(self.data)
    def __getitem__(self, idx):
        item = self.data[idx]
        out  = self.transforms({
            't1':    item['t1'],
            't1c':   item['t1c'],
            't2':    item['t2'],
            'flair': item['flair'],
            'label': item['label'],
        })
        return out['image'], out['label']

all_dicts      = build_ucsf_dicts(cfg.UCSF_CSV, cfg.UCSF_DIR)
train_d, val_d, _ = stratified_split(all_dicts)

train_seg_ds = SegDataset(train_d, get_ucsf_transforms('train'))
val_seg_ds   = SegDataset(val_d,   get_ucsf_transforms('val'))

train_seg_loader = torch.utils.data.DataLoader(
    train_seg_ds, batch_size=1, shuffle=True,
    num_workers=4, pin_memory=True, persistent_workers=True)
val_seg_loader   = torch.utils.data.DataLoader(
    val_seg_ds, batch_size=1, shuffle=False,
    num_workers=4, persistent_workers=True)

print(f"Seg train: {len(train_d)} patients")
print(f"Seg val:   {len(val_d)} patients")

# — Step 5: Loss and metric —————
def brats_label_to_channels(label):
    et = (label == 3).float()

```

```

tc = ((label == 1) | (label == 3)).float()
wt = (label >= 1).float()
return torch.cat([et, tc, wt], dim=1)

dice_loss = DiceLoss(sigmoid=True, squared_pred=True, reduction='mean')
dice_metric = DiceMetric(
    include_background=False,
    reduction=MetricReduction.MEAN,
    get_not_nans=True)

patience = 25 # more patience for sustained better-GPU run

print(f"\nStarting fine-tuning from epoch {start_epoch} to {FINETUNE_EPOCHS}...")
print(f"Best val Dice so far: {best_val_dice:.4f}")
print(f"Learning rate: {FINETUNE_LR}")
print(f"Patience: {patience}")
print("="*55)

# — Step 6: Training loop —————
for epoch in range(start_epoch, FINETUNE_EPOCHS):
    swinunetr.train()
    epoch_loss = 0.0
    optimizer.zero_grad()

    for i, (image, label) in enumerate(tqdm(
        train_seg_loader,
        desc=f"Ep {epoch+1}/{FINETUNE_EPOCHS} [train]",
        leave=False)):
        image = image.to(cfg.DEVICE)
        label = label.to(cfg.DEVICE)
        targets = brats_label_to_channels(label)
        logits = swinunetr(image)
        loss = dice_loss(logits, targets) / ACCUMULATION_STEPS
        loss.backward()
        if ((i+1) % ACCUMULATION_STEPS == 0) or \
            ((i+1) == len(train_seg_loader)):
            optimizer.step()
            optimizer.zero_grad()
            epoch_loss += loss.item() * ACCUMULATION_STEPS

    scheduler.step()
    avg_train_loss = epoch_loss / len(train_seg_loader)

```

```

# — Validation —————
swinunetr.eval()
dice_metric.reset()
with torch.no_grad():
    for image, label in tqdm(
        val_seg_loader,
        desc=f"Ep {epoch+1}/{FINETUNE_EPOCHS} [val]",
        leave=False):
        image = image.to(cfg.DEVICE)
        label = label.to(cfg.DEVICE)
        targets = brats_label_to_channels(label)
        logits = swinunetr(image)
        preds = (torch.sigmoid(logits) > cfg.SEG_THRESHOLD).float()
        dice_metric(y_pred=preds, y=targets)

val_dice, _ = dice_metric.aggregate()
mean_val_dice = val_dice.mean().item()

history['train_loss'].append(avg_train_loss)
history['val_dice'].append(mean_val_dice)

print(f"Epoch {epoch+1:3d} | "
      f"Train loss: {avg_train_loss:.4f} | "
      f"Val Dice: {mean_val_dice:.4f}")

# — Save best if improved —————
if mean_val_dice > best_val_dice:
    best_val_dice = mean_val_dice
    no_improve = 0
    torch.save({
        'epoch': epoch,
        'state_dict': swinunetr.state_dict(),
        'optimizer': optimizer.state_dict(),
        'scheduler': scheduler.state_dict(),
        'val_dice': best_val_dice,
        'best_val_dice': best_val_dice,
        'no_improve': no_improve,
        'history': history,
    }, FINETUNE_CKPT)
    print(f" ✓ Best model saved (val Dice: {best_val_dice:.4f})")
else:

```

```

no_improve += 1
print(f" No improve: {no_improve}/{patience}")
if no_improve >= patience:
    print(f"\nEarly stop at epoch {epoch+1}")
    break

# — Save latest every epoch —————
torch.save({
    'epoch':          epoch,
    'state_dict':      swinunetr.state_dict(),
    'optimizer':       optimizer.state_dict(),
    'scheduler':       scheduler.state_dict(),
    'val_dice':        mean_val_dice,
    'best_val_dice':   best_val_dice,
    'no_improve':      no_improve,
    'history':         history,
}, LATEST_CKPT)

torch.cuda.empty_cache()
gc.collect()

# — Step 7: Reload best and verify —————
print("\nReloading best fine-tuned weights...")
ckpt = torch.load(FINETUNE_CKPT, map_location=cfg.DEVICE, weights_only=False)
swinunetr.load_state_dict(ckpt['state_dict'])
swinunetr.eval()
print(f"Loaded epoch {ckpt['epoch']} – Val Dice {ckpt['val_dice']:.4f}")

def compute_dice(pred, true):
    pred, true = pred.astype(bool), true.astype(bool)
    u = pred.sum() + true.sum()
    return 1.0 if u == 0 else 2.0*(pred & true).sum() / u

sample = train_d[0]
trans = get_ucsf_transforms('val')
out = trans({
    't1':    sample['t1'],
    't1c':   sample['t1c'],
    't2':    sample['t2'],
    'flair': sample['flair'],
    'label': sample['label'],
})

```

```
image = out['image']
label = out['label'].squeeze().numpy()

with torch.no_grad():
    logits = swinunetr(image.unsqueeze(0).to(cfg.DEVICE))
    preds = torch.sigmoid(logits).squeeze(0).cpu().numpy()

wt_pred = preds[2] > cfg.SEG_THRESHOLD
wt_true = label >= 1
print(f"\nSanity check on {sample['patient_id']}:")
print(f"  WT Dice:          {compute_dice(wt_pred, wt_true):.4f}")
print(f"  WT pred coverage: {wt_pred.mean()*100:.2f}%")
print(f"  WT true coverage: {wt_true.mean()*100:.2f}%")
print(f"\nLatest checkpoint: {LATEST_CKPT}")
print(f"Best checkpoint:    {FINETUNE_CKPT}")
print(f"✓ Ready for next session or cache rebuild")
```

```

No improve: 18/25
Epoch 93 | Train loss: 0.1751 | Val Dice: 0.8707
No improve: 19/25
Epoch 94 | Train loss: 0.1756 | Val Dice: 0.8797
No improve: 20/25
Epoch 95 | Train loss: 0.1742 | Val Dice: 0.8804
No improve: 21/25
Epoch 96 | Train loss: 0.1755 | Val Dice: 0.8802
No improve: 22/25
Epoch 97 | Train loss: 0.1750 | Val Dice: 0.8691
No improve: 23/25
Epoch 98 | Train loss: 0.1736 | Val Dice: 0.8846
✓ Best model saved (val Dice: 0.8846)
Epoch 99 | Train loss: 0.1743 | Val Dice: 0.8800
No improve: 1/25
Epoch 100 | Train loss: 0.1737 | Val Dice: 0.8802
No improve: 2/25

```

```

Reloading best fine-tuned weights...
Loaded epoch 97 – Val Dice 0.8846

```

```

Sanity check on UCSF-PDGM-179:

```

```

WT Dice:          0.9277
WT pred coverage: 3.63%
WT true coverage: 3.84%

```

```

Latest checkpoint: /content/drive/MyDrive/checkpoints/swinunetr_ucsf_latest.pt
Best checkpoint:   /content/drive/MyDrive/checkpoints/swinunetr_ucsf_finetuned.pt
✓ Ready for next session or cache rebuild

```

```

print(cfg.UCSF_DIR)
print(cfg.UCSF_CSV)
import os

print(os.path.exists(cfg.UCSF_DIR))
print(os.path.exists(cfg.UCSF_CSV))

```

```

/content/UCSF-PDGM-v5
/content/drive/MyDrive/UCSF-PDGM-metadata_v5.csv
True
True

```

```

# =====
# CELL 10 – Loss functions

```

```

# =====
import torch.nn as nn

IDH_W = torch.tensor([2.5]).to(cfg.DEVICE)
GRADE_W = torch.tensor([92/395]).to(cfg.DEVICE)
MGMT_W = torch.tensor([0.395]).to(cfg.DEVICE)

print(f"IDH_W: {IDH_W.item():.4f}")
print(f"GRADE_W: {GRADE_W.item():.4f}")
print(f"MGMT_W: {MGMT_W.item():.4f}")

idh_crit = nn.BCEWithLogitsLoss(pos_weight=IDH_W)
grade_crit = nn.BCEWithLogitsLoss(pos_weight=GRADE_W)
mgmt_crit = nn.BCEWithLogitsLoss(pos_weight=MGMT_W)

print(f"grade_crit pos_weight: {grade_crit.pos_weight.item():.4f}")

TASK_W = {'idh':1.0, 'grade':1.0, 'mgmt':0.8}

def classification_loss(preds, labels):
    loss = 0.0
    idh_valid = ~torch.isnan(labels['idh'])
    if idh_valid.any():
        loss += TASK_W['idh'] * idh_crit(
            preds['idh'][idh_valid], labels['idh'][idh_valid].float())
    grade_valid = ~torch.isnan(labels['grade'])
    if grade_valid.any():
        loss += TASK_W['grade'] * grade_crit(
            preds['grade'][grade_valid], labels['grade'][grade_valid].float())
    mgmt_valid = ~torch.isnan(labels['mgmt'])
    if mgmt_valid.any():
        loss += TASK_W['mgmt'] * mgmt_crit(
            preds['mgmt'][mgmt_valid], labels['mgmt'][mgmt_valid].float())
    return loss

print("Loss functions defined – all weights confirmed correct")

IDH_W: 2.5000
GRADE_W: 0.2329
MGMT_W: 0.3950
grade_crit pos_weight: 0.2329
Loss functions defined – all weights confirmed correct

```

```
# CHECK ONCE BEFORE CREATING A NEW CACHE – NOT IMP BUT DO IT ANYWAY
print(sum(p.sum().item() for p in swinunetr.parameters()))
```

```
-34969.759021766484
```

```
# DELETING FILES BEFORE CACHE CONSTRUCTION
```

```
import shutil
```

```
# Full feature cache (Drive)
```

```
CACHE_DIR = '/content/drive/MyDrive/checkpoints/feature_cache_3d_ucsf_finetuned'
```

```
if os.path.exists(CACHE_DIR):
```

```
    shutil.rmtree(CACHE_DIR)
```

```
os.makedirs(CACHE_DIR)
```

```
# Slim cache, both copies
```

```
SLIM_CACHE_LOCAL = '/content/feature_cache_slim_ucsf'
```

```
SLIM_CACHE_DRIVE = '/content/drive/MyDrive/checkpoints/feature_cache_slim_ucsf'
```

```
for d in [SLIM_CACHE_LOCAL, SLIM_CACHE_DRIVE]:
```

```
    if os.path.exists(d):
```

```
        shutil.rmtree(d)
```

```
# =====
# CELL 11 – Precompute feature cache
# Stores [768] encoder features + morph + confidence per patient
# Tiny: ~3KB per patient, ~1.5MB total for all 487
# =====
```

```
import gc
```

```
from tqdm import tqdm
```

```
def precompute_feature_cache(model, csv_path, ucsf_dir, cache_dir, n_runs=cfg.TTA_RUNS):
```

```
    all_dicts = build_ucsf_dicts(csv_path, ucsf_dir)
```

```
    transforms = get_ucsf_transforms('val')
```

```
    model.eval()
```

```
    computed, skipped, failed = 0, 0, 0
```

```
    for item in tqdm(all_dicts, desc="Precomputing 3D features"):
```

```
        pid = item['patient_id']
```

```
        path = os.path.join(cache_dir, f"{pid}.pt")
```

```
        if os.path.exists(path):
```

```
            skipped += 1
```

```
            continue
```

```

try:
    sample = transforms({
        't1':item['t1'], 't1c':item['t1c'],
        't2':item['t2'], 'flair':item['flair'],
        'label':item['label'],
    })
    image = sample['image']
    enc_feat = extract_encoder_features(model, image)
    mean_pred, conf_map, _ = tta_predict(model, image, n_runs)
    conf_score = torch.tensor([conf_map.mean().item()])
    morph = morphology_to_tensor(extract_morphological_features(mean_pred))
    torch.save({
        'patient_id': pid,
        'encoder': enc_feat,
        'morph': morph,
        'conf_score': conf_score,
        'mean_pred': mean_pred,
        'label': sample['label'],
        'idh': torch.tensor([item['idh']], dtype=torch.float32),
        'grade': torch.tensor([item['grade']], dtype=torch.float32),
        'mgmt': torch.tensor([item['mgmt']], dtype=torch.float32),
    }, path)
    computed += 1
    torch.cuda.empty_cache(); gc.collect()
except Exception as e:
    print(f"Failed {pid}: {e}")
    failed += 1
    continue
print(f"\nComputed: {computed} Skipped: {skipped} Failed: {failed}")

```

```
precompute_feature_cache(swinunetr, cfg.UCSF_CSV, cfg.UCSF_DIR, CACHE_DIR)
```

```

/usr/local/lib/python3.12/dist-packages/monai/utils/deprecate_utils.py:321: FutureWarning: monai.transforms.spatial.dictionary
warn_deprecated(argname, msg, warning_category)

```

Patients loaded: 487

Patients skipped: 14

IDH: 391 wt / 96 mut

Grade: 92 low / 395 high

MGMT: 112 neg / 297 pos

Precomputing 3D features: 100%|██████████| 487/487 [27:57<00:00, 3.44s/it]

Computed: 487 Skipped: 0 Failed: 0

```

# =====
# CELL 11B – Recompute morphology at threshold 0.3 (robust version)
# =====

cfg.SEG_THRESHOLD = 0.3

from tqdm import tqdm
import time

CACHE_DIR = '/content/drive/MyDrive/checkpoints/feature_cache_3d_ucsf_finetuned'
def list_cache_files():
    for attempt in range(5):
        try:
            return [f for f in os.listdir(CACHE_DIR) if f.endswith('.pt')]
        except OSError:
            print("Listing failed, remounting...")
            drive.mount('/content/drive', force_remount=True)
            time.sleep(3)
    raise RuntimeError("Could not list cache dir")

files = list_cache_files()
print(f"Recomputing morphology for {len(files)} patients "
      f"at threshold {cfg.SEG_THRESHOLD}...")

failed = []
for fname in tqdm(files, desc="Recomputing morphology"):
    path = os.path.join(CACHE_DIR, fname)

    for attempt in range(4):
        try:
            d = torch.load(path, map_location='cpu', weights_only=False)
            d['morph'] = morphology_to_tensor(
                extract_morphological_features(d['mean_pred']))
            torch.save(d, path)
            break # success
        except OSError as e:
            if attempt < 3:
                drive.mount('/content/drive', force_remount=True)
                time.sleep(3)
            else:

```

```

print(f"\nFailed permanently: {fname}: {e}")
failed.append(fname)

print(f"\nDone. Failed: {len(failed)}")
if failed:
    print("Failed files:", failed[:10])

Recomputing morphology for 487 patients at threshold 0.3...
Recomputing morphology: 1%|          | 4/487 [00:10<22:00, 2.73s/it]Mounted at /content/drive
Recomputing morphology: 100%|██████████| 487/487 [22:52<00:00, 2.82s/it]
Done. Failed: 0

```

```

UTSW_DIR = '/content/UTSW-Glioma'
UTSW_TSV = '/content/drive/MyDrive/UTSW_Glioma_Metadata-2-1.tsv'
UTSW_CACHE_DIR = '/content/drive/MyDrive/checkpoints/feature_cache_3d_utsw'

# Verify all three exist before running cache build
print(os.path.exists(UTSW_DIR)) # should be True
print(os.path.exists(UTSW_TSV)) # should be True
os.makedirs(UTSW_CACHE_DIR, exist_ok=True)

# Quick sanity check on one patient
test_pid = 'BT0212'
test_folder = os.path.join(UTSW_DIR, test_pid)
for fname in ['brain_t1_ants.nii.gz', 'brain_t1ce_ants.nii.gz',
              'brain_t2_ants.nii.gz', 'brain_fl_ants.nii.gz',
              'tumorseg_resampled.nii.gz']:
    exists = os.path.exists(os.path.join(test_folder, fname))
    print(f" {fname}: {exists}")

```

```

True
True
brain_t1_ants.nii.gz: True
brain_t1ce_ants.nii.gz: True
brain_t2_ants.nii.gz: True
brain_fl_ants.nii.gz: True
tumorseg_resampled.nii.gz: True

```

```
import gc
```

```

# =====
# CELL 11C – Precompute UTSW feature cache (cross-dataset)
# =====
os.makedirs(UTSW_CACHE_DIR, exist_ok=True)

# Verify weights before touching UTSW cache
checksum = sum(p.sum().item() for p in swinunetr.parameters())
print(f"Pre-UTSW-cache checksum: {checksum:.2f}")
assert abs(checksum - (-34969.76)) < 1.0, "WRONG WEIGHTS – stop"

def precompute_utsw_cache(model, tsv_path, utsw_dir, cache_dir,
                          n_runs=cfg.TTA_RUNS):
    all_dicts = build_utsw_dicts(tsv_path, utsw_dir)
    # Reuse UCSF val transforms – same preprocessing pipeline
    transforms = get_ucsf_transforms('val')
    model.eval()
    computed, skipped, failed = 0, 0, 0

    for item in tqdm(all_dicts, desc="Precomputing UTSW features"):
        pid = item['patient_id']
        path = os.path.join(cache_dir, f"{pid}.pt")
        if os.path.exists(path):
            skipped += 1
            continue
        try:
            sample = transforms({
                't1': item['t1'], 't1c': item['t1c'],
                't2': item['t2'], 'flair': item['flair'],
                'label': item['label'],
            })
            image = sample['image']
            enc_feat = extract_encoder_features(model, image)
            mean_pred, conf_map, _ = tta_predict(model, image, n_runs)
            conf_score = torch.tensor([conf_map.mean().item()])
            morph = morphology_to_tensor(
                extract_morphological_features(mean_pred))
            torch.save({
                'patient_id': pid,
                'encoder': enc_feat,
                'morph': morph,
                'conf_score': conf_score,
                'mean_pred': mean_pred,

```

```

        'label': sample['label'],
        'idh': torch.tensor([item['idh']], dtype=torch.float32),
        'grade': torch.tensor([item['grade']], dtype=torch.float32),
        'mgmt': torch.tensor([item['mgmt']], dtype=torch.float32),
    }, path)
    computed += 1
    torch.cuda.empty_cache(); gc.collect()
except Exception as e:
    print(f"Failed {pid}: {e}")
    failed += 1

```

```

print(f"\nComputed: {computed} Skipped: {skipped} Failed: {failed}")
n = len([f for f in os.listdir(cache_dir) if f.endswith('.pt')])
print(f"Total UTSW cache files: {n}")

```

```
precompute_utsw_cache(swinunetr, UTSW_TSV, UTSW_DIR, UTSW_CACHE_DIR)
```

```

/usr/local/lib/python3.12/dist-packages/monai/transforms/spatial/dictionary.py:320: FutureWarning: monai.transforms.spatial.dictionary
warn_deprecated(argname, msg, warning_category)
Pre-UTSW-cache checksum: -34969.76
Using 362 manually refined patients
Patients loaded: 362
Patients skipped: 0
IDH: 256 wt / 103 mut
Grade: 124 low / 236 high
MGMT: 84 neg / 53 pos (note: 225 missing)
Precomputing UTSW features: 100%|██████████| 362/362 [20:24<00:00, 3.38s/it]
Computed: 346 Skipped: 16 Failed: 0
Total UTSW cache files: 362

```

```
FINETUNE_CKPT = os.path.join(cfg.CKPT_DIR, 'swinunetr_ucsf_finetuned.pt')
```

```

ckpt = torch.load(FINETUNE_CKPT, map_location='cpu', weights_only=False)
print(f"Current best checkpoint: epoch {ckpt['epoch']}, Val Dice {ckpt['val_dice']:.4f}")

```

```
Current best checkpoint: epoch 97, Val Dice 0.8846
```

```

# CHECK BEFORE SLIM CACHE -----
all_dicts = build_ucsf_dicts(cfg.UCSF_CSV, cfg.UCSF_DIR)
cached = {f.replace('.pt', '') for f in os.listdir(CACHE_DIR) if f.endswith('.pt')}
all_dicts = [d for d in all_dicts if d['patient_id'] in cached]

```

```
_, _, test_d = stratified_split(all_dicts)

dices = []
for item in test_d:
    d = torch.load(os.path.join(CACHE_DIR, f"{item['patient_id']}.pt"),
                    map_location='cpu', weights_only=False)
    mp, lab = d['mean_pred'].numpy(), d['label'].squeeze().numpy()
    wt_pred, wt_true = mp[2] > 0.3, lab >= 1
    u = wt_pred.sum() + wt_true.sum()
    dices.append(2*(wt_pred & wt_true).sum()/u if u > 0 else 1.0)
dices = np.array(dices)
print(f"Mean: {dices.mean():.4f}  Median: {np.median(dices):.4f}")
```

Patients loaded: 487

Patients skipped: 14

IDH: 391 wt / 96 mut

Grade: 92 low / 395 high

MGMT: 112 neg / 297 pos

Mean: 0.8655 Median: 0.9204

CELL 12

```
import shutil, os
from tqdm import tqdm
import torch
```

Delete old slim cache

```
SLIM_CACHE = '/content/feature_cache_slim_ucsf'
```

```
if os.path.exists(SLIM_CACHE):
```

```
    shutil.rmtree(SLIM_CACHE)
```

```
os.makedirs(SLIM_CACHE)
```

```
print("Old slim cache deleted")
```

Rebuild from new full cache

```
CACHE_DIR = '/content/drive/MyDrive/checkpoints/feature_cache_3d_ucsf_finetuned'
```

```
files = [f for f in os.listdir(CACHE_DIR) if f.endswith('.pt')]
```

```
print(f"Rebuilding from {len(files)} full cache files...")
```

```
failed = []
```

```
for fname in tqdm(files, desc="Building slim cache"):
```

```
    try:
```

```

        d = torch.load(os.path.join(CACHE_DIR, fname),
                        map_location='cpu', weights_only=False)
        torch.save({
            'patient_id': d['patient_id'],
            'encoder': d['encoder'],
            'morph': d['morph'],
            'conf_score': d['conf_score'],
            'idh': d['idh'],
            'grade': d['grade'],
            'mgmt': d['mgmt'],
        }, os.path.join(SLIM_CACHE, fname))
    except Exception as e:
        print(f"Failed {fname}: {e}")
        failed.append(fname)

print(f"Done. Failed: {len(failed)}")

# Verify
import numpy as np
max_vals = []
for fname in os.listdir(SLIM_CACHE):
    if not fname.endswith('.pt'):
        continue
    d = torch.load(os.path.join(SLIM_CACHE, fname),
                    map_location='cpu', weights_only=False)
    max_vals.append(d['morph'].max().item())

max_vals = np.array(max_vals)
print(f"\nAfter rebuild:")
print(f"Morph max - mean: {max_vals.mean():.3f}  max: {max_vals.max():.3f}")
print(f"Files above 15: {(max_vals > 15).sum()}")
if (max_vals > 15).sum() == 0:
    print("All clean - safe to train")
else:
    print("Still dirty - check full cache")

# Copy fresh slim cache to Drive as the new backup
import time
SLIM_CACHE_DRIVE = '/content/drive/MyDrive/checkpoints/feature_cache_slim_ucsf'
if os.path.exists(SLIM_CACHE_DRIVE):
    shutil.rmtree(SLIM_CACHE_DRIVE)
    print("Removed stale Drive backup")

```

```
print("Copying fresh slim cache to Drive...")
start = time.time()
shutil.copytree(SLIM_CACHE, SLIM_CACHE_DRIVE)
print(f"Done in {time.time()-start:.1f}s - {len(os.listdir(SLIM_CACHE_DRIVE))} files on Drive")
```

```
Old slim cache deleted
Rebuilding from 487 full cache files...
Building slim cache: 100%|██████████| 487/487 [09:37<00:00, 1.19s/it]
Done. Failed: 0
```

```
After rebuild:
Morph max - mean: 11.148 max: 12.594
Files above 15: 0
All clean - safe to train
Copying fresh slim cache to Drive...
Done in 4.4s - 487 files on Drive
```

```
import shutil, time, os

SLIM_CACHE_DRIVE = '/content/drive/MyDrive/checkpoints/feature_cache_slim_ucsf'
SLIM_CACHE_LOCAL = '/content/feature_cache_slim_ucsf'

if not os.path.exists(SLIM_CACHE_LOCAL):
    print("Copying slim cache from Drive...")
    start = time.time()
    shutil.copytree(SLIM_CACHE_DRIVE, SLIM_CACHE_LOCAL)
    print(f"Done in {time.time()-start:.1f}s")
else:
    print(f"Already local - {len(os.listdir(SLIM_CACHE_LOCAL))} files")

SLIM_CACHE = SLIM_CACHE_LOCAL
```

```
# Copy fresh slim cache to Drive as the new backup
import time
SLIM_CACHE_DRIVE = '/content/drive/MyDrive/checkpoints/feature_cache_slim_ucsf'
if os.path.exists(SLIM_CACHE_DRIVE):
    shutil.rmtree(SLIM_CACHE_DRIVE)
    print("Removed stale Drive backup")

print("Copying fresh slim cache to Drive...")
start = time.time()
```

```
shutil.copytree(SLIM_CACHE, SLIM_CACHE_DRIVE)
print(f"Done in {time.time()-start:.1f}s - {len(os.listdir(SLIM_CACHE_DRIVE))} files on Drive")
```

```
Removed old Drive slim cache
Copying clean slim cache to Drive...
Done in 3.7 seconds
Files on Drive: 487
```

```
# cell 13 with seed
import random
import numpy as np
import pandas as pd

def set_seed(seed=42):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False

SEEDS = [0, 1, 2, 3, 4]
all_results = []

for seed in SEEDS:
    print(f"\n{'='*55}\n SEED {seed}\n{'='*55}")
    set_seed(seed)

    classifier = BrainTumorClassifier().to(cfg.DEVICE)
    classifier, history = train_classifier(
        classifier, SLIM_CACHE, cfg.UCSF_CSV, cfg.UCSF_DIR, tag=f"seed{seed}")

    _, _, test_loader = build_loaders(SLIM_CACHE, cfg.UCSF_CSV, cfg.UCSF_DIR)
    metrics = evaluate_classifier(classifier, test_loader)

    row = {'seed': seed}
    for task in ['idh', 'grade', 'mgmt']:
        if task in metrics:
            row[f'{task}_auc'] = metrics[task]['auc']
            row[f'{task}_acc'] = metrics[task]['accuracy']
            row[f'{task}_f1'] = metrics[task]['f1']
            row[f'{task}_sens'] = metrics[task]['sensitivity']
```

```
        row[f'{task}_spec'] = metrics[task]['specificity']
    all_results.append(row)

results_df = pd.DataFrame(all_results)
print("\n\nPer-seed results:")
print(results_df.round(3))

print(f"\n{'='*55}\n SUMMARY (mean  $\pm$  std across {len(SEEDS)} seeds)\n{'='*55}")
for task in ['idh', 'grade', 'mgmt']:
    col = f'{task}_auc'
    if col in results_df.columns:
        m, s = results_df[col].mean(), results_df[col].std()
        lo, hi = results_df[col].min(), results_df[col].max()
        print(f"{task.upper():6s} AUC: {m:.3f}  $\pm$  {s:.3f} (range {lo:.3f}–{hi:.3f})")
```

```

✓ Best saved (val 0.8928)
Ep  2 | Train 1.1068 acc 0.816 | Val 0.8213 acc 0.903
✓ Best saved (val 0.8213)
Ep  3 | Train 1.0608 acc 0.831 | Val 0.8503 acc 0.889
Ep  4 | Train 1.0718 acc 0.805 | Val 0.9047 acc 0.847
Ep  5 | Train 1.0437 acc 0.819 | Val 0.8903 acc 0.917
Ep  6 | Train 1.0231 acc 0.845 | Val 0.8442 acc 0.889
Ep  7 | Train 1.0345 acc 0.831 | Val 0.8842 acc 0.889
Ep  8 | Train 1.0188 acc 0.825 | Val 0.8121 acc 0.944
✓ Best saved (val 0.8121)
Ep  9 | Train 0.9939 acc 0.828 | Val 0.8245 acc 0.931
Ep 10 | Train 0.9995 acc 0.831 | Val 0.8493 acc 0.903
Ep 11 | Train 0.9885 acc 0.851 | Val 0.8686 acc 0.889
Ep 12 | Train 1.0098 acc 0.822 | Val 0.8223 acc 0.903
Ep 13 | Train 0.9953 acc 0.840 | Val 0.8204 acc 0.903
Ep 14 | Train 0.9686 acc 0.845 | Val 0.8030 acc 0.917
✓ Best saved (val 0.8030)
Ep 15 | Train 0.9576 acc 0.840 | Val 0.7995 acc 0.903
✓ Best saved (val 0.7995)
Ep 16 | Train 0.9705 acc 0.837 | Val 0.8431 acc 0.889
Ep 17 | Train 0.9699 acc 0.854 | Val 0.8250 acc 0.931
Ep 18 | Train 0.9552 acc 0.848 | Val 0.8057 acc 0.903
Ep 19 | Train 0.9875 acc 0.813 | Val 0.8151 acc 0.889
Ep 20 | Train 0.9358 acc 0.851 | Val 0.7971 acc 0.931
✓ Best saved (val 0.7971)
Ep 21 | Train 0.9558 acc 0.831 | Val 0.7977 acc 0.903
Ep 22 | Train 0.9664 acc 0.866 | Val 0.7738 acc 0.931
✓ Best saved (val 0.7738)

```

```

# =====
# CELL 13 – Classifier training with early stopping
# =====

```

```

class CachedDataset(torch.utils.data.Dataset):
    def __init__(self, files):
        self.files = files
    def __len__(self):
        return len(self.files)
    def __getitem__(self, idx):
        d = torch.load(self.files[idx], map_location='cpu',
                        weights_only=False)
        return {
            'encoder': d['encoder'],
            'morph': d['morph'],
            'conf_score': d['conf_score'].view(1),

```

```

        'idh': d['idh'], 'grade': d['grade'], 'mgmt': d['mgmt'],
        'patient_id': d['patient_id'],
    }

```

```

def build_loaders(cache_dir, csv_path, ucsf_dir):
    all_dicts = build_ucsf_dicts(csv_path, ucsf_dir)
    cached_pids = {f.replace('.pt','') for f in os.listdir(cache_dir)
                    if f.endswith('.pt')}
    all_dicts = [d for d in all_dicts if d['patient_id'] in cached_pids]
    print(f"Patients with cache: {len(all_dicts)}")

    train_d, val_d, test_d = stratified_split(all_dicts)
    def paths(dicts):
        return [os.path.join(cache_dir, f"{d['patient_id']}.pt")
                for d in dicts]

    print(f"Split: {len(train_d)} train / {len(val_d)} val / {len(test_d)} test")

    tl = torch.utils.data.DataLoader(
        CachedDataset(paths(train_d)), batch_size=config.BATCH_SIZE,
        shuffle=True, num_workers=0, pin_memory=True)
    vl = torch.utils.data.DataLoader(
        CachedDataset(paths(val_d)), batch_size=config.BATCH_SIZE,
        shuffle=False, num_workers=0)
    tel = torch.utils.data.DataLoader(
        CachedDataset(paths(test_d)), batch_size=1,
        shuffle=False, num_workers=0)
    return tl, vl, tel

def train_classifier(clf_model, cache_dir, csv_path, ucsf_dir,
                    epochs=config.CLF_EPOCHS, patience=20,
                    tag=config.GATE_MODE):
    train_loader, val_loader, _ = build_loaders(cache_dir, csv_path, ucsf_dir)

    optimizer = torch.optim.AdamW(clf_model.parameters(),
                                   lr=config.LR, weight_decay=1e-5)
    scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
        optimizer, T_max=epochs)

    history = {'train_loss':[], 'val_loss':[], 'train_acc':[], 'val_acc':[]}

```

```

best_ckpt = os.path.join(cfg.CKPT_DIR, f'clf_best_{tag}.pth')
latest_ckpt = os.path.join(cfg.CKPT_DIR, f'clf_latest_{tag}.pth')

best_val, no_improve = float('inf'), 0

# fresh start
for ck in [best_ckpt, latest_ckpt]:
    if os.path.exists(ck):
        os.remove(ck)
print(f"Training variant {tag} from scratch")

for epoch in range(epochs):
    clf_model.train()
    ep_loss, ep_corr, ep_tot = 0.0, 0, 0
    for batch in tqdm(train_loader, desc=f"Ep {epoch+1}/{epochs}",
                       leave=False):
        enc = batch['encoder'].to(cfg.DEVICE)
        mor = batch['morph'].to(cfg.DEVICE)
        conf = batch['conf_score'].to(cfg.DEVICE)
        lbl = {k: batch[k].to(cfg.DEVICE)
                for k in ['idh', 'grade', 'mgmt']}
        optimizer.zero_grad()
        preds = clf_model(enc, mor, conf)
        loss = classification_loss(preds, lbl)
        loss.backward()
        optimizer.step()
        ep_loss += loss.item()
        v = ~torch.isnan(lbl['idh'])
        if v.any():
            p = torch.sigmoid(preds['idh'][v])>0.5
            t = lbl['idh'][v]>0.5
            ep_corr += (p==t).sum().item(); ep_tot += v.sum().item()
    scheduler.step()
    tr_loss = ep_loss/len(train_loader)
    tr_acc = ep_corr/ep_tot if ep_tot>0 else 0.0

    clf_model.eval()
    v_loss, v_corr, v_tot = 0.0, 0, 0
    with torch.no_grad():
        for batch in val_loader:
            enc = batch['encoder'].to(cfg.DEVICE)
            mor = batch['morph'].to(cfg.DEVICE)

```

```

        conf = batch['conf_score'].to(cfg.DEVICE)
        lbl = {k: batch[k].to(cfg.DEVICE)
                for k in ['idh', 'grade', 'mgmt']}
        preds = clf_model(enc, mor, conf)
        v_loss += classification_loss(preds, lbl).item()
        vv = ~torch.isnan(lbl['idh'])
        if vv.any():
            p = torch.sigmoid(preds['idh'][vv])>0.5
            t = lbl['idh'][vv]>0.5
            v_corr += (p==t).sum().item(); v_tot += vv.sum().item()
    va_loss = v_loss/len(val_loader)
    va_acc = v_corr/v_tot if v_tot>0 else 0.0

    for k, val in [('train_loss', tr_loss), ('val_loss', va_loss),
                  ('train_acc', tr_acc), ('val_acc', va_acc)]:
        history[k].append(val)

    print(f"Ep {epoch+1:3d} | Train {tr_loss:.4f} acc {tr_acc:.3f} "
          f"| Val {va_loss:.4f} acc {va_acc:.3f}")

    torch.save({'epoch': epoch, 'model_state': clf_model.state_dict(),
               'history': history, 'best_val_loss': best_val}, latest_ckpt)

    if va_loss < best_val:
        best_val, no_improve = va_loss, 0
        torch.save({'epoch': epoch, 'model_state': clf_model.state_dict(),
                   'val_loss': best_val, 'history': history}, best_ckpt)
        print(f" ✓ Best saved (val {best_val:.4f})")
    else:
        no_improve += 1
        if no_improve >= patience:
            print(f"\nEarly stop at epoch {epoch+1}")
            break

    print(f"\nDone variant {tag}. Best val: {best_val:.4f}")
    return clf_model, history

# Run (gate mode D, the proposed variant):
classifier, history = train_classifier(
    classifier, SLIM_CACHE, cfg.UCSF_CSV, cfg.UCSF_DIR)

```

```
✓ Best saved (val 0.7752)
Ep 13 | Train 0.9891 acc 0.816 | Val 0.7978 acc 0.917
Ep 14 | Train 0.9783 acc 0.848 | Val 0.8248 acc 0.903
Ep 15 | Train 0.9558 acc 0.860 | Val 0.7821 acc 0.903
Ep 16 | Train 0.9669 acc 0.840 | Val 0.8576 acc 0.889
Ep 17 | Train 0.9331 acc 0.837 | Val 0.7800 acc 0.917
Ep 18 | Train 0.9278 acc 0.851 | Val 0.7648 acc 0.931
✓ Best saved (val 0.7648)
Ep 19 | Train 0.9284 acc 0.857 | Val 0.8115 acc 0.903
Ep 20 | Train 0.9679 acc 0.840 | Val 0.7587 acc 0.931
✓ Best saved (val 0.7587)
Ep 21 | Train 0.9220 acc 0.845 | Val 0.8001 acc 0.931
Ep 22 | Train 0.9740 acc 0.866 | Val 0.8018 acc 0.917
Ep 23 | Train 0.9025 acc 0.860 | Val 0.7761 acc 0.917
Ep 24 | Train 0.9184 acc 0.857 | Val 0.7568 acc 0.917
✓ Best saved (val 0.7568)
Ep 25 | Train 0.9763 acc 0.840 | Val 0.7733 acc 0.917
Ep 26 | Train 0.9315 acc 0.866 | Val 0.7648 acc 0.931
Ep 27 | Train 0.9118 acc 0.860 | Val 0.7626 acc 0.931
Ep 28 | Train 0.8799 acc 0.854 | Val 0.8372 acc 0.889
Ep 29 | Train 0.9258 acc 0.875 | Val 0.7658 acc 0.931
```

```

Ep 51 | Train 0.8565 acc 0.872 | Val 0.7575 acc 0.931
Ep 52 | Train 0.8567 acc 0.883 | Val 0.7541 acc 0.931
Ep 53 | Train 0.8555 acc 0.878 | Val 0.7547 acc 0.931
Ep 54 | Train 0.8674 acc 0.869 | Val 0.7562 acc 0.931
Ep 55 | Train 0.8583 acc 0.886 | Val 0.7560 acc 0.931
Ep 56 | Train 0.8361 acc 0.889 | Val 0.7560 acc 0.931
Ep 57 | Train 0.8490 acc 0.883 | Val 0.7558 acc 0.931
Ep 58 | Train 0.8524 acc 0.869 | Val 0.7558 acc 0.931
Ep 59 | Train 0.8234 acc 0.883 | Val 0.7558 acc 0.931
Ep 60 | Train 0.8559 acc 0.883 | Val 0.7558 acc 0.931

```

Done variant D. Best val: 0.7491

```

# =====
# CELL 14 – Run full ablation study (variants A, B, C, D)
# =====

def run_ablation(cache_dir, csv_path, ucsf_dir):
    """Trains all 4 gate variants and returns their histories."""
    results = {}
    for mode in ['A', 'B', 'C', 'D']:
        print(f"\n{'='*55}\n  ABLATION VARIANT {mode}\n{'='*55}")
        model = BrainTumorClassifier(gate_mode=mode).to(cfg.DEVICE)
        _, hist = train_classifier(model, cache_dir, csv_path, ucsf_dir,
                                   tag=mode)
        results[mode] = hist
    return results

ablation_results = run_ablation(SLIM_CACHE, cfg.UCSF_CSV, cfg.UCSF_DIR)

```

```

Ep 20 | Train 0.9277 acc 0.865 | Val 0.7865 acc 0.951
Ep 21 | Train 0.9324 acc 0.857 | Val 0.7633 acc 0.903
Ep 22 | Train 0.9330 acc 0.854 | Val 0.7624 acc 0.931
Ep 23 | Train 0.9490 acc 0.843 | Val 0.8145 acc 0.903
Ep 24 | Train 0.9588 acc 0.851 | Val 0.7622 acc 0.903
Ep 25 | Train 0.9234 acc 0.860 | Val 0.7905 acc 0.903
Ep 26 | Train 0.9012 acc 0.848 | Val 0.7729 acc 0.903
Ep 27 | Train 0.9183 acc 0.860 | Val 0.7630 acc 0.903
Ep 28 | Train 0.9264 acc 0.848 | Val 0.8163 acc 0.903
Ep 29 | Train 0.9012 acc 0.863 | Val 0.7678 acc 0.917
Ep 30 | Train 0.9037 acc 0.869 | Val 0.7706 acc 0.917
Ep 31 | Train 0.8771 acc 0.875 | Val 0.7533 acc 0.917
✓ Best saved (val 0.7533)
Ep 32 | Train 0.9202 acc 0.854 | Val 0.7657 acc 0.917
Ep 33 | Train 0.9060 acc 0.857 | Val 0.7579 acc 0.931
Ep 34 | Train 0.9011 acc 0.845 | Val 0.7820 acc 0.917
Ep 35 | Train 0.8884 acc 0.883 | Val 0.7568 acc 0.917
Ep 36 | Train 0.8825 acc 0.872 | Val 0.7554 acc 0.917
Ep 37 | Train 0.8740 acc 0.886 | Val 0.7620 acc 0.931
Ep 38 | Train 0.8855 acc 0.860 | Val 0.7588 acc 0.917
Ep 39 | Train 0.8948 acc 0.889 | Val 0.7615 acc 0.917
Ep 40 | Train 0.8625 acc 0.866 | Val 0.7542 acc 0.917
Ep 41 | Train 0.8614 acc 0.889 | Val 0.7546 acc 0.917
Ep 42 | Train 0.8750 acc 0.860 | Val 0.7664 acc 0.917
Ep 43 | Train 0.8398 acc 0.889 | Val 0.7539 acc 0.917
Ep 44 | Train 0.8661 acc 0.875 | Val 0.7566 acc 0.917
Ep 45 | Train 0.8600 acc 0.892 | Val 0.7568 acc 0.931
Ep 46 | Train 0.8471 acc 0.869 | Val 0.7511 acc 0.917
✓ Best saved (val 0.7511)
Ep 47 | Train 0.8577 acc 0.892 | Val 0.7555 acc 0.931
Ep 48 | Train 0.8453 acc 0.875 | Val 0.7554 acc 0.917
Ep 49 | Train 0.8377 acc 0.875 | Val 0.7594 acc 0.903
Ep 50 | Train 0.8543 acc 0.889 | Val 0.7532 acc 0.917
Ep 51 | Train 0.8514 acc 0.869 | Val 0.7533 acc 0.917
Ep 52 | Train 0.8638 acc 0.872 | Val 0.7558 acc 0.917
Ep 53 | Train 0.8275 acc 0.886 | Val 0.7551 acc 0.917
Ep 54 | Train 0.8297 acc 0.872 | Val 0.7550 acc 0.917
Ep 55 | Train 0.8291 acc 0.889 | Val 0.7538 acc 0.917
Ep 56 | Train 0.8314 acc 0.889 | Val 0.7549 acc 0.917
Ep 57 | Train 0.8433 acc 0.880 | Val 0.7546 acc 0.917
Ep 58 | Train 0.8371 acc 0.889 | Val 0.7547 acc 0.917
Ep 59 | Train 0.8407 acc 0.883 | Val 0.7547 acc 0.917
Ep 60 | Train 0.8545 acc 0.880 | Val 0.7547 acc 0.917

```

Done variant D Best val: 0.7511

```
CACHE_DIR = '/content/drive/MyDrive/checkpoints/feature_cache_3d_ucsf_finetuned'
```

```
import os
n = len([f for f in os.listdir(CACHE_DIR) if f.endswith('.pt')])
print(f"Full cache files: {n}")
assert n == 487, "Full cache incomplete or path wrong"
```

```
Full cache files: 487
```

```
# =====
# CELL 15 – Evaluation (inline display, classification + segmentation)
# =====
```

```
import matplotlib.pyplot as plt
```

```
def compute_dice(pred, true):
    pred, true = pred.astype(bool), true.astype(bool)
    u = pred.sum()+true.sum()
    return 1.0 if u==0 else 2.0*(pred&true).sum()/u

def compute_hd95(pred, true):
    pred, true = pred.astype(bool), true.astype(bool)
    if pred.sum()==0 or true.sum()==0:
        return float('nan')
    ps = pred & ~binary_erosion(pred)
    ts = true & ~binary_erosion(true)
    d1 = distance_transform_edt(~true)[ps]
    d2 = distance_transform_edt(~pred)[ts]
    return max(np.percentile(d1,95), np.percentile(d2,95))
```

```
def evaluate_classifier(clf_model, test_loader):
    clf_model.eval()

    res = {t: {'probs': [], 'labels': []}
           for t in ['idh', 'grade', 'mgmt']}

    with torch.no_grad():
        for batch in tqdm(test_loader, desc="Classification eval"):

            enc = batch['encoder'].to(cfg.DEVICE)
```

```
mor = batch['morph'].to(cfg.DEVICE)
conf = batch['conf_score'].to(cfg.DEVICE)

preds = clf_model(enc, mor, conf)

for t in ['idh', 'grade', 'mgmt']:
    res[t]['probs'].append(
        torch.sigmoid(preds[t]).item()
    )
    res[t]['labels'].append(
        batch[t].item()
    )

metrics = {}

names = {
    'idh': 'IDH Status',
    'grade': 'WHO Grade',
    'mgmt': 'MGMT'
}

for t in ['idh', 'grade', 'mgmt']:

    p = np.array(res[t]['probs'])
    l = np.array(res[t]['labels'])

    valid = ~np.isnan(l)

    p = p[valid]
    l = l[valid]

    if len(l) == 0:
        continue

    pb = (p >= 0.5).astype(int)

    try:
        auc = roc_auc_score(l, p)
    except:
        auc = float('nan')

    tn, fp, fn, tp = confusion_matrix(
```

```
l,
pb,
labels=[0, 1]
).ravel()

sensitivity = tp / (tp + fn) if (tp + fn) > 0 else 0
specificity = tn / (tn + fp) if (tn + fp) > 0 else 0

metrics[t] = {
    'accuracy': accuracy_score(l, pb),
    'precision': precision_score(
        l,
        pb,
        zero_division=0
    ),
    'recall': recall_score(
        l,
        pb,
        zero_division=0
    ),
    'f1': f1_score(
        l,
        pb,
        zero_division=0
    ),
    'auc': auc,

    'sensitivity': sensitivity,
    'specificity': specificity,

    'tp': tp,
    'tn': tn,
    'fp': fp,
    'fn': fn,

    'probs': p,
    'labels': l,
    'preds': pb,

    'name': names[t],
    'n': len(l)
}
```

```
return metrics
```

```
# Quick sanity check on one file
import random
sample_file = random.choice([f for f in os.listdir(CACHE_DIR) if f.endswith('.pt')])
d = torch.load(os.path.join(CACHE_DIR, sample_file), map_location='cpu', weights_only=False)
mp = d['mean_pred'].numpy()
lab = d['label'].squeeze().numpy()
wt_pred = mp[2] > 0.3
wt_true = lab >= 1
inter = (wt_pred & wt_true).sum()
dice = 2 * inter / (wt_pred.sum() + wt_true.sum() + 1e-6)
print(f"Sample {sample_file} - WT Dice: {dice:.4f}")
```

```
CACHE_DIR = '/content/drive/MyDrive/checkpoints/feature_cache_3d_ucsf_finetuned'
print(f"Seg eval from: {CACHE_DIR}")
```

```
from scipy import ndimage
```

```
def keep_largest_component(mask):
    labeled, n = ndimage.label(mask)
    if n <= 1:
        return mask
    sizes = ndimage.sum(mask, labeled, range(1, n+1))
    largest = np.argmax(sizes) + 1
    return labeled == largest

def evaluate_segmentation(full_cache_dir, csv_path, ucsf_dir):
    all_dicts = build_ucsf_dicts(csv_path, ucsf_dir)
    cached = {f.replace('.pt', '') for f in os.listdir(full_cache_dir)
               if f.endswith('.pt')}
    all_dicts = [d for d in all_dicts if d['patient_id'] in cached]
    _, test_d = stratified_split(all_dicts)
    res = {r: {'dice': [], 'hd95': []} for r in ['ET', 'TC', 'WT']}
    for item in tqdm(test_d, desc="Segmentation eval"):
        try:
            d = torch.load(os.path.join(full_cache_dir,
                                         f"{item['patient_id']}.pt"),
                           map_location='cpu', weights_only=False)
```

```

except: continue
if 'mean_pred' not in d: continue
mp = d['mean_pred'].numpy(); lab = d['label'].squeeze().numpy()
for name,pred,true in [
    ('ET', mp[0]>cfg.SEG_THRESHOLD, lab==3),
    ('TC', mp[1]>cfg.SEG_THRESHOLD, (lab==1)|(lab==3)),
    ('WT', mp[2]>cfg.SEG_THRESHOLD, lab>=1)]:
    pred_clean = keep_largest_component(pred) # ← added line
    res[name]['dice'].append(compute_dice(pred_clean,true))
    res[name]['hd95'].append(compute_hd95(pred_clean,true))
seg = {}
for r in ['WT','TC','ET']:
    dv = [x for x in res[r]['dice'] if not np.isnan(x)]
    hv = [x for x in res[r]['hd95'] if not np.isnan(x)]
    seg[r] = {'dice':np.mean(dv) if dv else float('nan'),
              'hd95':np.mean(hv) if hv else float('nan'),
              'dice_std':np.std(dv) if dv else 0,
              'hd95_std':np.std(hv) if hv else 0,
              'n':len(dv)}
return seg

def display_all(metrics, seg, history):
    # Training curves
    fig,ax = plt.subplots(1,2,figsize=(14,5))
    ep = range(1,len(history['train_loss'])+1)
    ax[0].plot(ep,history['train_loss'],'b-o',ms=3,label='Train')
    ax[0].plot(ep,history['val_loss'],'r-o',ms=3,label='Val')
    ax[0].set_title('Loss',fontweight='bold'); ax[0].legend(); ax[0].grid(alpha=0.3)
    ax[1].plot(ep,history['train_acc'],'b-o',ms=3,label='Train')
    ax[1].plot(ep,history['val_acc'],'r-o',ms=3,label='Val')
    ax[1].set_title('Accuracy (IDH)',fontweight='bold'); ax[1].legend()
    ax[1].grid(alpha=0.3); ax[1].set_ylim([0,1])
    plt.suptitle('Training Curves',fontweight='bold'); plt.tight_layout(); plt.show()

    # Confusion matrices
    # Confusion matrices
    tasks = [t for t in ['idh','grade','mgmt'] if t in metrics]
    nm = {'idh': 'IDH\n(0=WT, 1=Mut)',
          'grade': 'Grade\n(0=Low, 1=High)',
          'mgmt': 'MGMT\n(0=Neg, 1=Pos)'}

```

```

fig, ax = plt.subplots(1, len(tasks), figsize=(6*len(tasks), 5))
if len(tasks) == 1:
    ax = [ax]

for a, t in zip(ax, tasks):
    m = metrics[t]

    # Force 2x2 regardless of collapse
    cm = confusion_matrix(m['labels'], m['preds'], labels=[0, 1])
    disp = ConfusionMatrixDisplay(
        confusion_matrix=cm,
        display_labels=['Neg', 'Pos'])
    disp.plot(ax=a, cmap='Blues', colorbar=False)
    a.set_title(nm[t], fontweight='bold')

    auc_str = f"{m['auc']:.3f}" if not np.isnan(m['auc']) else 'N/A'
    a.set_xlabel(
        f"Acc={m['accuracy']:.3f} F1={m['f1']:.3f} AUC={auc_str}",
        fontsize=10)

    # Annotate collapse if it happened
    unique_preds = np.unique(m['preds'])
    if len(unique_preds) == 1:
        a.set_title(
            nm[t] + f"\n△ COLLAPSE (all pred={unique_preds[0]})",
            fontweight='bold', color='red')

plt.suptitle('Confusion Matrices – UCSF-PDGM Test Set',
            fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

# ROC
fig,a=plt.subplots(figsize=(7,6))
col={'idh':'#2196F3','grade':'#4CAF50','mgmt':'#FF9800'}
for t,c in col.items():
    if t not in metrics or np.isnan(metrics[t]['auc']): continue
    m=metrics[t]; fpr, tpr, _ = roc_curve(m['labels'], m['probs'])
    a.plot(fpr, tpr, color=c, lw=2.5, label=f"{m['name']} ({m['auc']:.3f})")
    a.plot([0,1], [0,1], 'k--', lw=1, label='Random')
    a.set_title('ROC Curves', fontweight='bold'); a.legend(); a.grid(alpha=0.3)
    a.set_xlabel('FPR'); a.set_ylabel('TPR'); plt.tight_layout(); plt.show()

```

```

# Segmentation bars
regions=['WT','TC','ET']; cols=['#2196F3','#4CAF50','#FF9800']
dv=[seg[r]['dice'] for r in regions]; hv=[seg[r]['hd95'] for r in regions]
fig,ax=plt.subplots(1,2,figsize=(12,5))
ax[0].bar(regions,dv,color=cols,yerr=[seg[r]['dice_std'] for r in regions],
          capsize=5,edgecolor='black',lw=0.5)
ax[0].set_ylim([0,1.05]); ax[0].set_title('Dice (higher better)',fontweight='bold')
ax[0].axhline(0.8,color='r',ls='--',alpha=0.5); ax[0].grid(alpha=0.3,axis='y')
for i,v in enumerate(dv):
    if not np.isnan(v): ax[0].text(i,v+0.03,f'{v:.3f}',ha='center',fontweight='bold')
hv2=[h if not np.isnan(h) else 0 for h in hv]
ax[1].bar(regions,hv2,color=cols,capsize=5,edgecolor='black',lw=0.5)
ax[1].set_title('HD95 (lower better)',fontweight='bold'); ax[1].grid(alpha=0.3,axis='y')
for i,v in enumerate(hv):
    if not np.isnan(v): ax[1].text(i,hv2[i]+0.5,f'{v:.1f}',ha='center',fontweight='bold')
plt.suptitle('SwinUNETR Segmentation',fontweight='bold'); plt.tight_layout(); plt.show()

# Text summary
print("\n"+"="*55+"\n SUMMARY\n"+"="*55)
print("\nClassification:")

for t in ['idh', 'grade', 'mgmt']:    # ← 4 spaces indent
    if t not in metrics:
        continue
    m = metrics[t]
    auc = f"{m['auc']:.3f}" if not np.isnan(m['auc']) else 'N/A'
    print(f"\n{m['name']}")
    print(f" Accuracy      : {m['accuracy']:.3f}")
    print(f" Precision       : {m['precision']:.3f}")
    print(f" Recall          : {m['recall']:.3f}")
    print(f" F1              : {m['f1']:.3f}")
    print(f" AUC             : {auc}")
    print(f" Sensitivity      : {m['sensitivity']:.3f}")
    print(f" Specificity      : {m['specificity']:.3f}")
    print(f" TP={m['tp']} FP={m['fp']} TN={m['tn']} FN={m['fn']}")

print("\nSegmentation:")
for r in ['WT','TC','ET']:
    m=seg[r]; d=f"{m['dice']:.3f}" if not np.isnan(m['dice']) else 'N/A'
    h=f"{m['hd95']:.1f}" if not np.isnan(m['hd95']) else 'N/A'
    print(f" {r:<4} Dice={d} HD95={h}")

```

```

print("="*55)

# — Segmentation (deterministic, unaffected by classifier seed) —
seg = evaluate_segmentation(CACHE_DIR, cfg.UCSF_CSV, cfg.UCSF_DIR)

# — Aggregate classification summary across seeds (the reported numbers) —
print("="*55)
print(f"  AGGREGATE CLASSIFICATION SUMMARY (mean ± std, n={len(SEEDS)} seeds)")
print("="*55)
for task in ['idh', 'grade', 'mgmt']:
    col = f'{task}_auc'
    if col in results_df.columns:
        print(f"\n{task.upper()}")
        for metric in ['auc', 'acc', 'f1', 'sens', 'spec']:
            c = f'{task}_{metric}'
            m, s = results_df[c].mean(), results_df[c].std()
            print(f"  {metric:<12}: {m:.3f} ± {s:.3f}")

print("\nSegmentation:")
for r in ['WT', 'TC', 'ET']:
    m = seg[r]
    print(f"  {r:<4} Dice={m['dice']:.3f}  HD95={m['hd95']:.1f}")
print("="*55)

# — Visuals: pick one representative seed (closest to mean IDH AUC) for plots —
rep_seed = results_df.iloc[
    (results_df['idh_auc'] - results_df['idh_auc'].mean()).abs().argsort()[:1]
]['seed'].values[0]
print(f"\nPlots below are from seed {rep_seed} (closest to mean IDH AUC) — "
      f"see aggregate numbers above for the actual reported results.")

ckpt = torch.load(os.path.join(cfg.CKPT_DIR, f'clf_best_seed{rep_seed}.pth'),
                  map_location=cfg.DEVICE, weights_only=False)
classifier.load_state_dict(ckpt['model_state'])
latest = torch.load(os.path.join(cfg.CKPT_DIR, f'clf_latest_seed{rep_seed}.pth'),
                   map_location=cfg.DEVICE, weights_only=False)
_, _, test_loader = build_loaders(SLIM_CACHE, cfg.UCSF_CSV, cfg.UCSF_DIR)
metrics_rep = evaluate_classifier(classifier, test_loader)
display_all(metrics_rep, seg, latest['history'])

```



Sample UCSF-PDGM-0429_FU003d.pt – WT Dice: 0.9652
 Seg eval from: /content/drive/MyDrive/checkpoints/feature_cache_3d_ucsf_finetuned
 Patients loaded: 487
 Patients skipped: 14

IDH: 391 wt / 96 mut
 Grade: 92 low / 395 high
 MGMT: 112 neg / 297 pos
 Segmentation eval: 100%|██████████| 72/72 [01:41<00:00, 1.40s/it]

=====

AGGREGATE CLASSIFICATION SUMMARY (mean ± std, n=5 seeds)

=====

IDH

auc	: 0.917 ± 0.006
acc	: 0.889 ± 0.000
f1	: 0.714 ± 0.000
sens	: 0.714 ± 0.000
spec	: 0.931 ± 0.000

GRADE

auc	: 0.976 ± 0.001
acc	: 0.922 ± 0.012
f1	: 0.952 ± 0.008
sens	: 0.936 ± 0.008
spec	: 0.862 ± 0.034

MGMT

auc	: 0.427 ± 0.006
acc	: 0.532 ± 0.026
f1	: 0.652 ± 0.034
sens	: 0.609 ± 0.068
spec	: 0.329 ± 0.135

Segmentation:

WT	Dice=0.859	HD95=9.0
TC	Dice=0.758	HD95=6.3
ET	Dice=0.727	HD95=5.7

=====

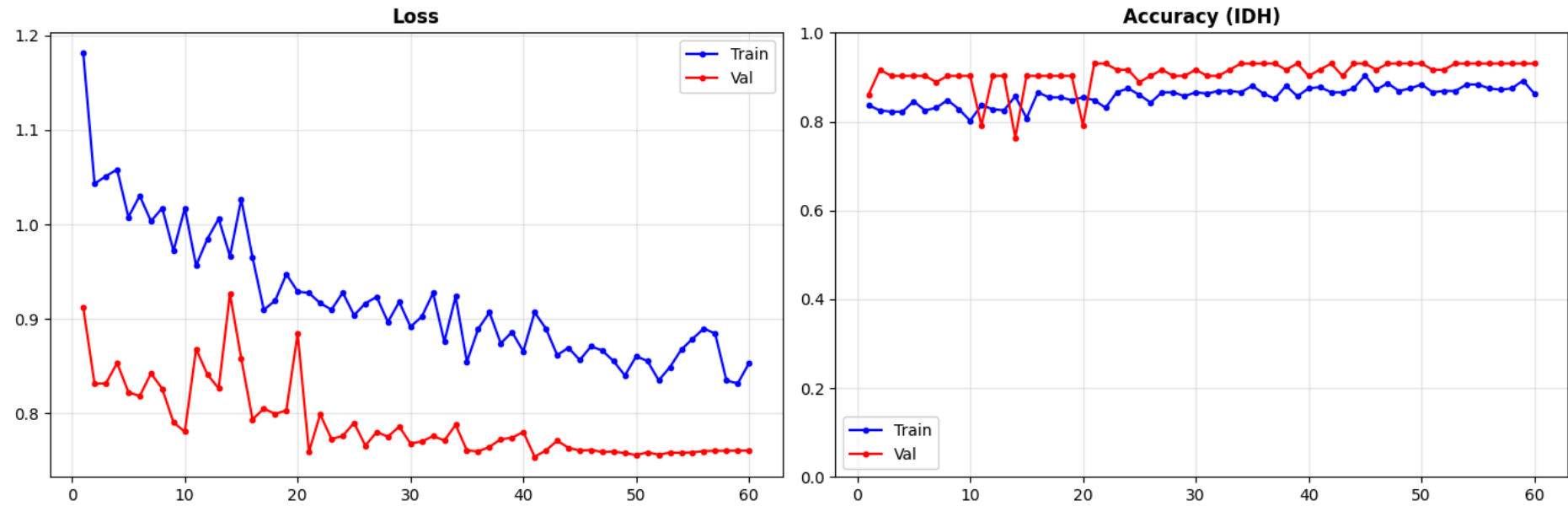
Plots below are from seed 1 (closest to mean IDH AUC) – see aggregate numbers above for the actual reported results.

Patients loaded: 487
 Patients skipped: 14

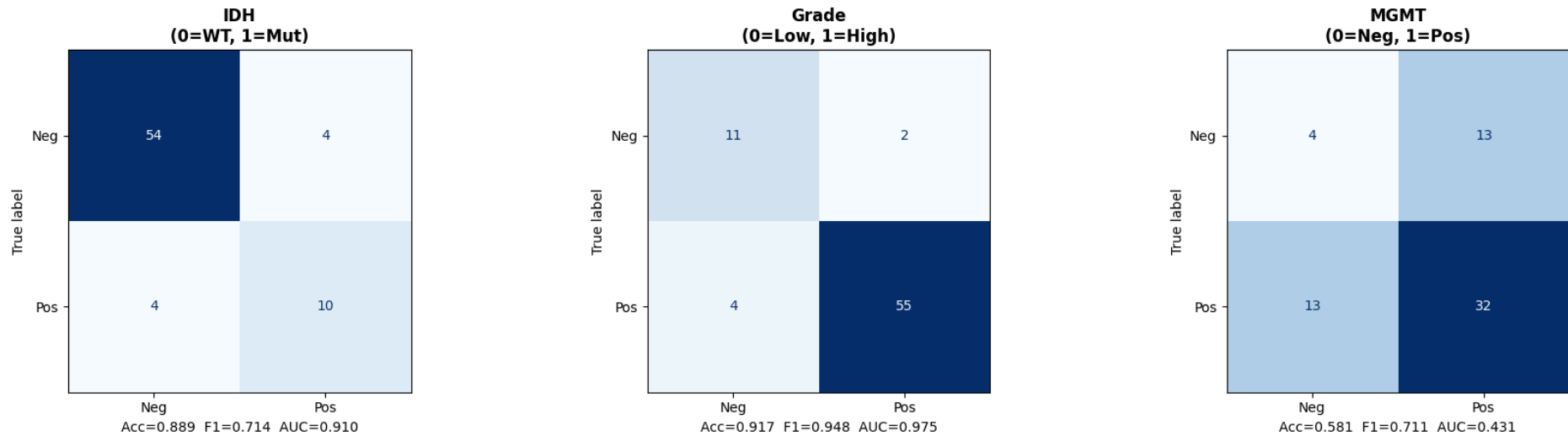
IDH: 391 wt / 96 mut
 Grade: 92 low / 395 high

MGMI: 112 neg / 29/ pos
Patients with cache: 487
Split: 343 train / 72 val / 72 test
Classification eval: 100%|██████████| 72/72 [00:00<00:00, 267.00it/s]

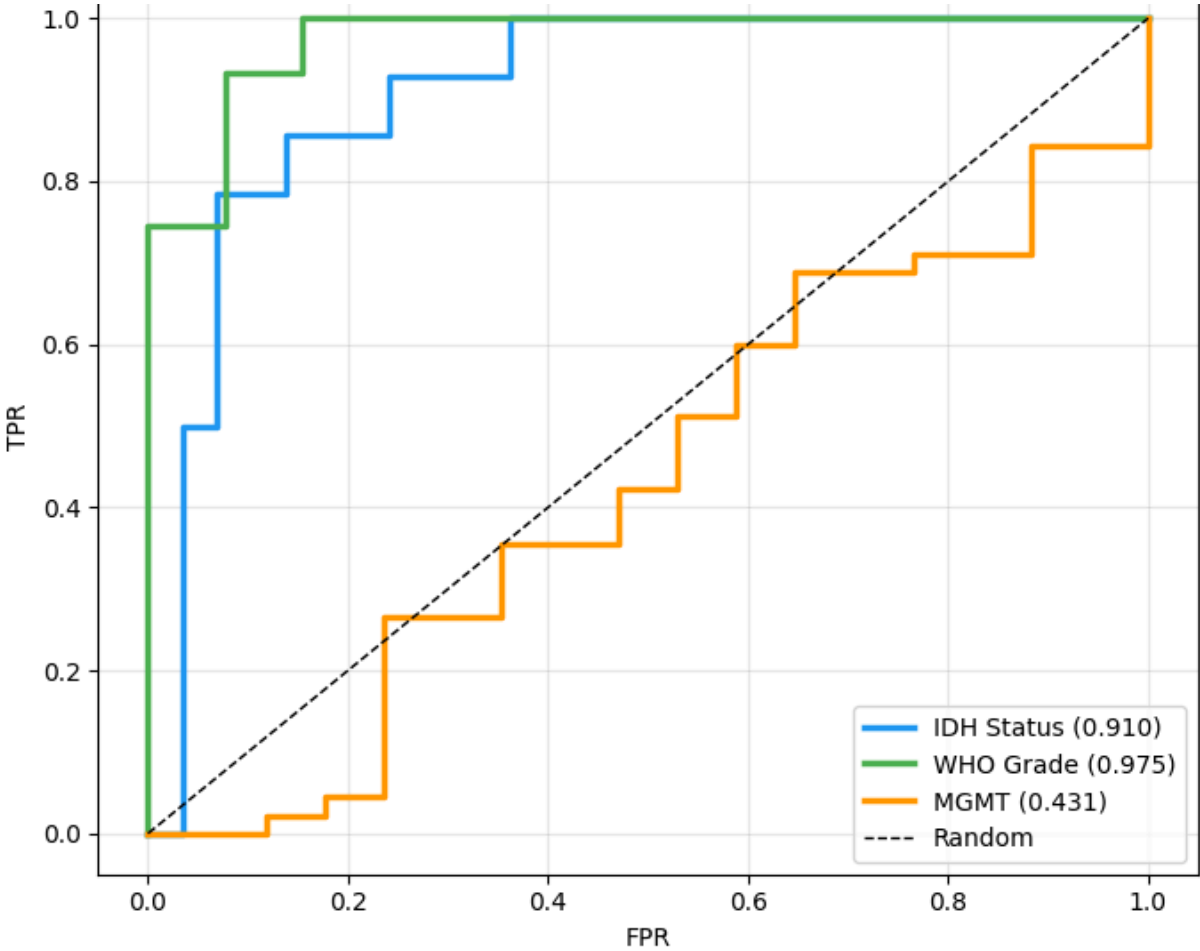
Training Curves



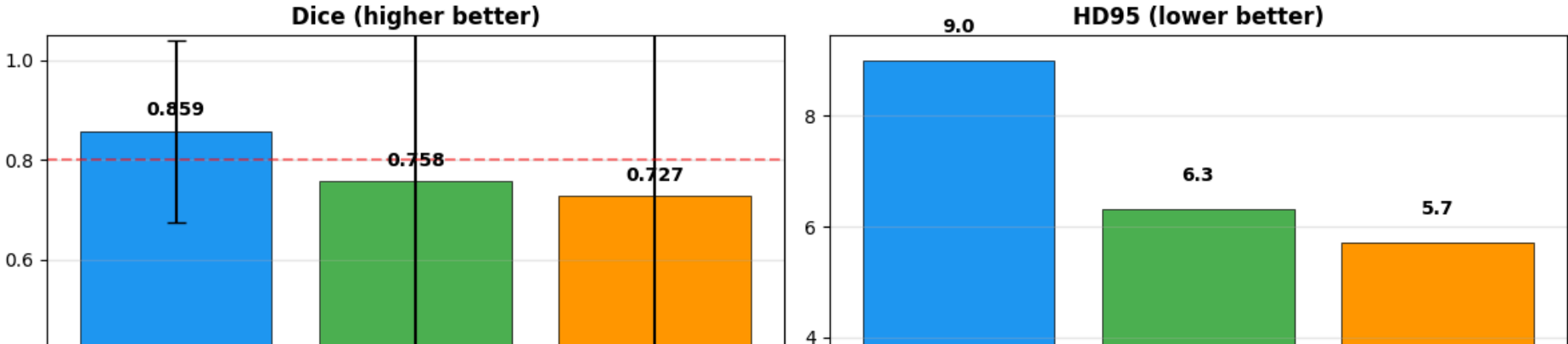
Confusion Matrices — UCSF-PDGM Test Set

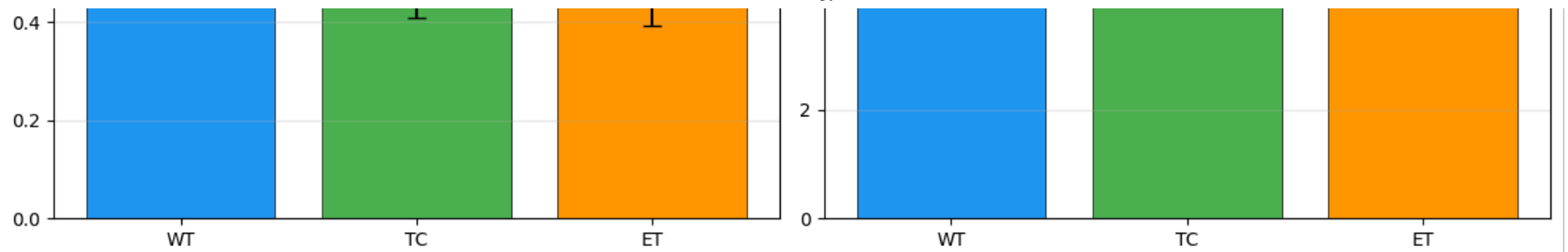


ROC Curves



SwinUNETR Segmentation





SUMMARY

Classification:

IDH Status

Accuracy : 0.889
 Precision : 0.714
 Recall : 0.714
 F1 : 0.714
 AUC : 0.910
 Sensitivity : 0.714
 Specificity : 0.931
 TP=10 FP=4 TN=54 FN=4

WHO Grade

Accuracy : 0.917
 Precision : 0.965
 Recall : 0.932
 F1 : 0.948
 AUC : 0.975
 Sensitivity : 0.932
 Specificity : 0.846
 TP=55 FP=2 TN=11 FN=4

MGMT

Accuracy : 0.581
 Precision : 0.711
 Recall : 0.711
 F1 : 0.711
 AUC : 0.431
 Sensitivity : 0.711
 Specificity : 0.235
 TP=32 FP=13 TN=4 FN=13

```

# Five-fold cross validation
from sklearn.model_selection import StratifiedKFold

def run_five_fold_cv(cache_dir, csv_path, ucsf_dir, seed=42):
    all_dicts = build_ucsf_dicts(csv_path, ucsf_dir)
    cached_pids = {f.replace('.pt', '') for f in os.listdir(cache_dir)
                    if f.endswith('.pt')}
    all_dicts = [d for d in all_dicts if d['patient_id'] in cached_pids]

    # Stratify on IDH label for balanced folds
    labels = np.array([d['idh'] if not np.isnan(d['idh']) else -1
                      for d in all_dicts])
    skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=seed)

    fold_results = []
    all_probs = {t: [] for t in ['idh', 'grade', 'mgmt']}
    all_labels = {t: [] for t in ['idh', 'grade', 'mgmt']}
    histories = []

    for fold, (train_idx, test_idx) in enumerate(skf.split(all_dicts, labels)):
        print(f"\n{' '*55}\n  FOLD {fold+1}/5\n{' '*55}")
        set_seed(seed + fold)

        train_dicts = [all_dicts[i] for i in train_idx]
        test_dicts = [all_dicts[i] for i in test_idx]

        def paths(dicts):
            return [os.path.join(cache_dir, f"{d['patient_id']}.pt")
                    for d in dicts]

        train_loader = torch.utils.data.DataLoader(
            CachedDataset(paths(train_dicts)),
            batch_size=cfg.BATCH_SIZE, shuffle=True, num_workers=0)
        test_loader = torch.utils.data.DataLoader(
            CachedDataset(paths(test_dicts)),
            batch_size=1, shuffle=False, num_workers=0)

        clf = BrainTumorClassifier().to(cfg.DEVICE)
        clf, history = train_classifier(
            clf, cache_dir, csv_path, ucsf_dir, tag=f"fold{fold+1}")
        histories.append(history)

```